3.5. Путь записи в Linux глубже

Страничный кеш · измененные страницы · fsync · WAL · blk-mq · NVMe · SSD

write()

приложение → ядро → кеш

Сохранность

измененные страницы → сброс → fsync

WAL

допись → групповой COMMIT → ответ

Накопитель

blk-mq → NVMe → FTL → NAND

Главный вопрос: когда успешная запись становится данными, которые переживут сбой?

Углубленная лекция после тем о виртуальной памяти и освобождении страниц

1. Три разных события

`write()` вернулся

Механизм

Различаем три события:

1. Ядро приняло данные.

2. Ввод-вывод на накопитель завершен.

3. Данные переживут сбой.

Это необходимо для понимания WAL, COMMIT и fsync().

Ядро приняло данные, накопитель завершил I/O и данные стали надежно сохраненными - разные этапы.

2. Почему Linux вообще буферизует запись

```
write(fd, buf, 4096)
↓
устройство
↓
ожидание
↓
возврат
```

Механизм

Наивный путь:

Каждый write() ждет накопитель.

Это дорого для мелких записей.

Поэтому Linux использует страничный кеш.

Буферизация позволяет write() завершиться до физической записи на накопитель.

3. Страничный кеш

```
read()
↓
страничный кеш
├── попадание → RAM
└── промах → накопитель
```

Механизм

Чтение: сначала страничный кеш.

Запись: данные меняются в кеше.

Накопитель обновляется позже.

Страничный кеш - управляемый ядром кеш файловых данных в RAM.

4. Почему страничный кеш полезен

`write()` вернулся

Механизм

Он дает:

кеширование чтения;

предварительное чтение;

объединение мелких записей;

асинхронную фоновую запись.

объединение мелких записей;

5. Буферизованная запись

```
буфер пользовательского пространства
↓
write()
↓
VFS
↓
файловая система
↓
страничный кеш
↓
скопировать данные
↓
пометить страницу измененной
↓
возврат
```

Механизм

Учебно:

То есть write() может завершиться до I/O на устройство.

Обычная запись в файл в Linux чаще всего проходит через страничный кеш.

6. Измененная страница

```
SSD:
A B C D

страничный кеш:
A B X D
```

Механизм

Страница изменена в RAM.

Пока фоновая запись не завершена, на накопителе остается прежняя версия.

Страничный кеш содержит более новую версию данных, чем накопитель.

7. Чистая страница

```
содержимое страничного кеша
=
содержимое накопителя
```

Механизм

После успешной фоновой записи страница становится чистой.

Под давлением на память ее проще освободить.

Чистая страница совпадает с данными накопителя и не требует предварительной записи.

8. Почему измененную страницу нельзя просто освободить

```
новые данные потеряны
```

Механизм

Если выбросить измененную страницу до записи:

новые данные будут потеряны.

Сначала требуется запись на накопитель.

Давление на память связывается со ввод-вывод накопителя.

9. Копирование при буферизованной записи

```
пользовательское пространство
↓
copy_from_user
↓
страничный кеш
```

Механизм

При обычной записи данные копируются из памяти процесса в страничный кеш.

Это дополнительная работа CPU, но ядро получает контроль над кешированием и фоновой записью.

Ядро копирует данные в собственный страничный кеш.

10. VFS

VFS

Механизм

VFS - единый интерфейс файловых операций.

Через него write() обращается к ext4, XFS и другим ФС.

VFS связывает дескрипторы, файлы и inode.

write() не знает, работает файл на ext4, XFS или другой файловой системе.

11. Файловый дескриптор

```
fd
↓
объект файла в ядре
↓
inode / состояние файловой системы
```

Механизм

Дескриптор fd - целое число, указывающее на открытый файл в ядре.

Через этот объект write() попадает в конкретную файловую систему.

Файловый дескриптор — не файл как байты.

12. Что происходит после изменения страниц

writeback

Механизм

Страничный кеш уже изменен.

Но когда и кто перенесет данные на накопитель?

Этим занимается фоновая запись.

Кто и когда отправляет измененные данные вниз?

13. Фоновая запись (`writeback`)

```
измененная страница
↓
фоновая запись ФС
↓
блочный ввод-вывод
↓
накопитель
```

Механизм

Он может запускаться:

фоново;

при накоплении измененных страниц;

при давлении на память;

по fsync;

Writeback — сброс измененных страниц страничного кеша на основной накопитель.

14. Объем измененных данных как очередь

```
объем измененных данных
=
очередь еще не выполненной записи на накопитель
```

Механизм

Приложение создает измененные страницы.

Накопитель записывает их с конечной скоростью.

Если приход быстрее обслуживания, очередь растет.

Измененные страницы образуют очередь работы, ожидающей записи на накопитель.

15. Почему очередь нельзя растить бесконечно

```
Dirty ↑
↓
в RAM накапливается все больше несброшенных данных
```

Механизм

Если накопитель отстает, объем несохраненных данных в RAM растет.

Поэтому ядро в определенный момент вынуждено замедлять пишущие задачи.

Измененные страницы занимают RAM.

16. Пороги измененных страниц

```
vm.dirty_background_ratio
vm.dirty_ratio
```

Механизм

Настройки Linux:

vm.dirty_background_ratio;

vm.dirty_ratio;

а также варианты порогов в байтах.

Конкретная политика зависит от ядра.

Linux имеет настройки вроде:

17. `balance_dirty_pages`

```
write()
↓
учет измененных страниц
↓
слишком много измененной памяти
↓
balance_dirty_pages
↓
задача засыпает / замедляется
```

Механизм

При избытке измененных страниц задача записи проходит через balance_dirty_pages и может ждать.

Теперь задержка write() зависит от того, успевает ли накопитель.

При превышении допустимого темпа записи ядро замедляет пишущую задачу.

18. Почему задержка записи бывает пилообразной

```
write()
→ RAM
→ быстро
```

Механизм

Пока есть запас измененных страниц, write() быстрый.

После достижения порога задача замедляется.

Когда фоновая запись освобождает запас, задержка падает.

Пока бюджет измененных страниц есть:

19. Почему это может не быть видно в профиле CPU Go

```
Go-обработчик
↓
write()
↓
замедление со стороны ядра
↓
ожидание
```

Механизм

Профиль CPU может не показать горячий участок: обработчик большую часть времени ожидает внутри ядра.

Время ожидания - не время выполнения CPU.

Горутина может ожидать завершения системного вызова, не нагружая CPU.

20. Как writeback связан с файловой системой

```
какая страница страничного кеша относится к какому inode
```

Механизм

Ядро знает, какая страница кеша относится к inode.

Файловая система решает, какие блоки выделить, как обновить экстенты и как соблюсти порядок записи метаданных.

Файловая система управляет размещением данных и изменением метаданных.

21. Данные и метаданные

```
данные файла
размер
временные метки
права доступа
запись каталога
отображение блоков
метаданные inode
```

Механизм

Есть:

Допись в конец может изменить и данные, и метаданные.

Долговечность должна учитывать оба уровня.

Файл — не только полезные данные.

22. Почему `fsync()` не равен «запиши эти байты»

```
измененные данные
+
метаданные, необходимые для чтения данных
+
журнал и порядок записи файловой системы
+
семантика сброса/FUA устройства
```

Механизм

Для сохранности файла fsync() может потребовать запись данных, нужных метаданных, журнала ФС и сброс кеша накопителя.

Поэтому он значительно дороже обычного write().

fsync(fd) должен закрыть обязательства по гарантированному сохранению файла.

23. `fsync()`

```
приложение
↓
fsync(fd)
↓
записать измененные данные
↓
записать необходимые метаданные
↓
дождаться завершения ввода-вывода
↓
завершить требуемый порядок сохранения
↓
возврат
```

Механизм

Учебно:

Вот это уже граница гарантированного сохранения прикладного уровня.

24. `write()` против `fsync()`

Событие	Что подтверждено
write() вернулся	ядро приняло данные согласно семантике системного вызова
фоновая запись завершена	ввод-вывод по пути записи завершен
fsync() вернулся	файловая система и ОС выполнили контракт гарантированного сохранения
подтверждение устройства	устройство подтвердило команду согласно ее гарантиям

Это не одна и та же точка.

25. `fdatasync()`

```
данные
+
метаданные, необходимые для чтения данных
```

Механизм

fdatasync() сохраняет данные и необходимые для их чтения метаданные.

Его контракт может быть уже, чем у fsync().

Но выигрыш в скорости не гарантирован.

fdatasync() ориентирован на данные и минимально необходимые метаданные.

26. `O_SYNC` и `O_DSYNC`

```
write()
↓
данные не могут просто остаться в обычной очереди измененных
страниц
↓
обязательства по синхронизации выполняются до возврата
```

Механизм

O_SYNC и O_DSYNC задают синхронную семантику записи.

Цена гарантированного сохранения оплачивается каждой соответствующей операцией write().

Можно открыть файл с синхронной семантикой.

27. `fsync()` решил проблему файла. Но транзакция — это уже другой уровень

```
write()
↓
страничный кеш
↓
fsync()
↓
граница гарантированного сохранения файла
```

Механизм

То есть приложение может потребовать:

Все необходимые изменения этого файла должны пережить сбой согласно контракту файловой системы и накопителя.

Кажется, дальше база данных просто должна вызвать fsync().

Но теперь меняем постановку задачи.

Транзакция обычно изменяет несколько независимых областей:

К этому месту мы уже умеем делать одну важную вещь:

28. Специально ломаем прямую запись страниц данных

```
счет A: -100
счет B: +100
```

Механизм

Пусть есть перевод денег:

Самая очевидная реализация:

Теперь машина падает в неприятный момент.

Накопитель уже гарантированно сохранил:

Но еще не успел сохранить:

Условно значения лежат на двух разных страницах данных:

29. Почему нельзя просто синхронно записывать все страницы

```
страница 17
страница 821
страница 44103
страница индекса 91
страница каталога 4
```

Механизм

Но посмотрим на цену.

Одна транзакция может изменить:

Критический путь COMMIT превращается в:

Чем больше транзакция, тем больше данных нужно синхронно довести до постоянного состояния перед ответом клиенту.

Для OLTP это дорого.

Теоретически можно построить сложный протокол вокруг самих страниц данных.

30. Какой объект удобно делать гарантированно сохраненным

```
транзакция изменила A, B, C
↓
сформировать данные для восстановления
↓
дописать их в последовательный журнал
```

Механизм

Учебно:

Теперь на критическом пути нужно гарантированно сохранить:

Вот из этой потребности естественно появляется WAL.

Вместо немедленной синхронной записи всех конечных страниц данных можно сначала сохранить компактную информацию, достаточную для восстановления изменений.

31. WAL: что это на этом уровне

```
транзакция меняет состояние в памяти
↓
формируется запись WAL
↓
WAL дописывается в конец журнала
↓
необходимая часть WAL гарантированно сохраняется
↓
БД может подтвердить гарантированный COMMIT
```

Механизм

На текущем уровне достаточно такой модели:

WAL хранит информацию, необходимую для восстановления изменений, и должен становиться гарантированно сохраненным раньше тех страниц данных, которые эти изменения отражают.

Учебная цепочка:

Страницы данных при этом могут жить своим более медленным жизненным циклом.

WAL — Write-Ahead Log, журнал упреждающей записи.

32. Почему именно Write-Ahead

```
сначала WAL
↓
потом страница данных
```

Механизм

Ключевое правило:

Точнее:

Это и есть правило упреждающей записи (write-ahead rule).

Слово Ahead здесь буквально означает:

Прежде чем измененная страница данных может безопасно стать гарантированно сохраненной, WAL, необходимый для восстановления этой версии страницы, уже должен быть гарантированно сохранен.

33. Что это дает при сбое

```
на накопителе есть измененная страница
+
есть информация для восстановления
```

Механизм

Если соответствующий WAL уже гарантированно сохранен:

Если страница данных еще не успела записаться:

После сбоя БД может использовать журнал, чтобы привести файлы данных к правильному состоянию.

Получаем главное разделение:

Допустим страница данных успела попасть на накопитель.

34. Важная точность: страница данных не обязана ждать COMMIT

```
COMMIT
↓
только потом можно писать страницы данных
```

Механизм

Это не универсальное правило WAL-систем.

Более точная модель:

То есть страница данных в некоторых архитектурах может попасть на накопитель еще до COMMIT транзакции, если правило упреждающей записи выполнено.

Как конкретная СУБД после сбоя различает зафиксированные и незафиксированные изменения — отдельная тема ее протокола восстановления.

Для PostgreSQL мы разберем это позже.

Здесь легко построить слишком простую модель:

35. Где появляется гарантированный COMMIT

```
изменения транзакции
↓
записи WAL
↓
запись COMMIT
↓
сброс WAL до границы гарантированного сохранения
↓
ACK клиенту
```

Механизм

Теперь другой вопрос:

Недостаточно, чтобы запись COMMIT существовала только в RAM.

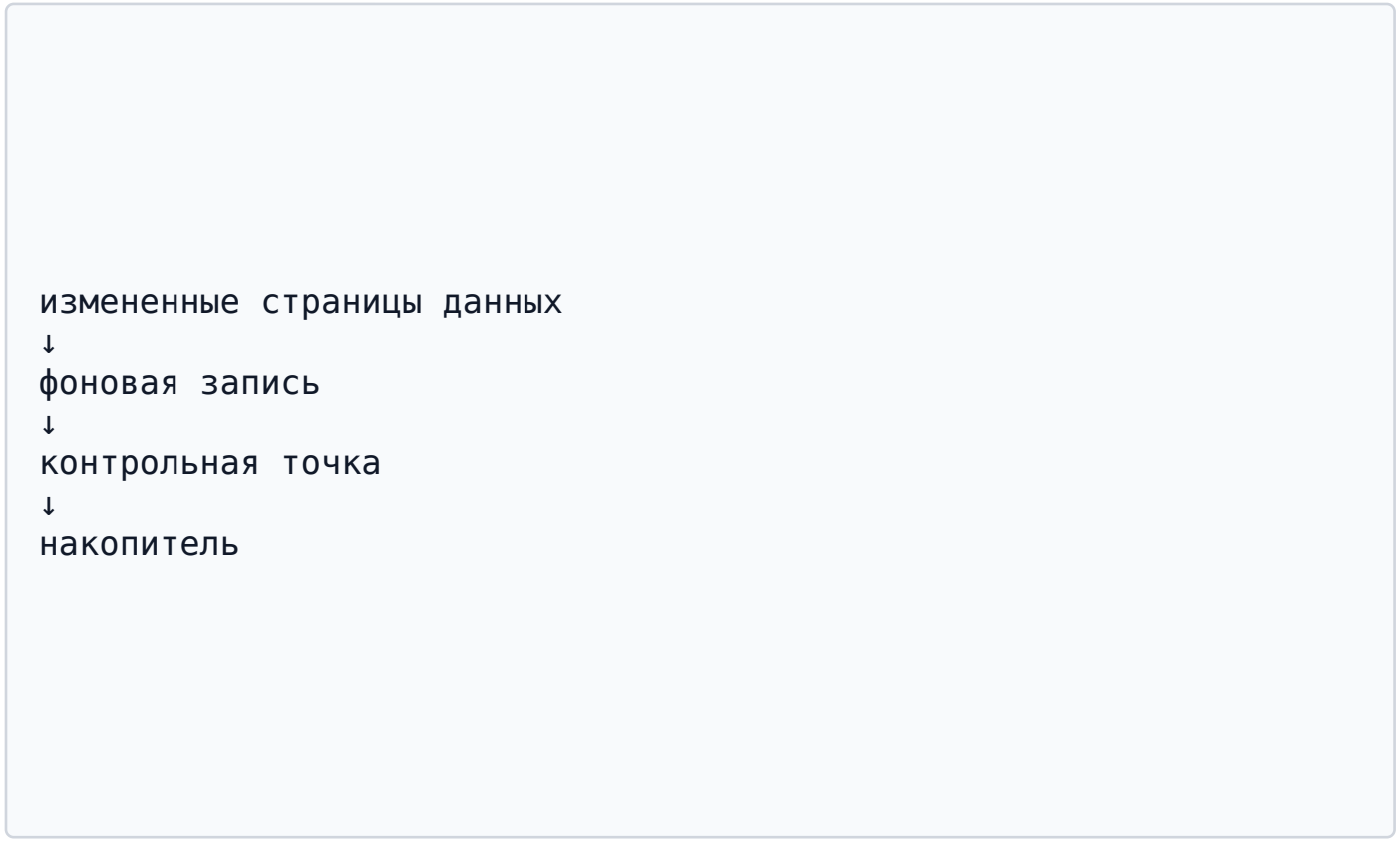
Нужна цепочка:

Главная формула:

Гарантированный COMMIT можно подтвердить после того, как информация WAL, необходимая для восстановления этого COMMIT после сбоя, стала гарантированно сохраненной.

В какой момент клиенту можно сказать COMMIT OK?

36. Почему страницы данных можно не синхронизировать на каждом COMMIT



Механизм

Страницы данных могут оставаться измененными в памяти:

Получаем принципиальное разделение:

Именно поэтому каждый COMMIT не обязан синхронно записывать весь набор случайных страниц данных.

После гарантированного сохранения нужной части WAL у транзакции уже есть информация для восстановления.

37. Сбой после ACK, но до записи страницы данных

```
COMMIT в WAL гарантированно сохранен
↓
клиент получил COMMIT OK
↓
страница данных на накопителе все еще старая
↓
машина падает
```

Механизм

Сценарий:

После перезапуска:

Точный алгоритм восстановления зависит от СУБД.

На высоком уровне это REDO:

38. WAL — не просто журнал аудита

```
гарантированного сохранения
+
восстановления после сбоя
+
правильного порядка записи
+
возможности откладывать запись страниц данных
```

Механизм

Плохая модель:

Слишком слабо.

WAL нужен ради:

Это часть протокола хранения.

Не просто история событий.

WAL — файл, куда база логирует, что она делала.

39. Почему WAL обычно ориентирован на допись в конец

случайные страницы данных
разбросаны по файлам БД

Механизм

Исходная проблема:

Поэтому WAL естественно организован как журнал с дописью в конец:

Накопителю проще эффективно обслуживать такой поток, чем множество мелких случайных гарантированно сохраняемых обновлений.

На критическом пути мы хотим заменить это на:

40. Но теперь каждый COMMIT требует сброса

много случайных гарантированно сохраняемых
записей страниц данных

Механизм

Мы заменили:

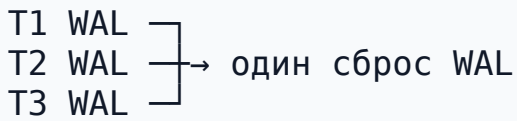
на:

Наивно:

При большом TPS накопитель быстро упрется в задержку таких сбросов.

Но сама граница гарантированного сохранения все еще дорогая.

41. Групповой COMMIT появляется сам



Механизм

Можно:

После завершения одной операции гарантированного сохранения все транзакции, чьи записи COMMIT уже попали в гарантированно сохраненную часть WAL, можно подтвердить клиентам.

Это групповой COMMIT (групповой COMMIT).

Дорогая граница гарантированного сохранения распределяется между несколькими транзакциями.

Пусть несколько транзакций почти одновременно дошли до COMMIT.

42. Почему задержка COMMIT теперь зависит от `fsync()`

```
транзакция
↓
допись WAL
↓
ожидание гарантированного сохранения нужной позиции WAL
↓
fsync / эквивалентная операция
↓
накопитель
↓
ACK
```

Механизм

Цепочка уже прозрачна:

Максимальные GB/s SSD здесь почти ничего не объясняют.

Поэтому маленькая транзакция может менять всего сотню байт, но ее задержка все равно зависит от задержки гарантированного сброса.

43. WAL базы данных и журнал файловой системы — разные журналы

страницы базы данных
транзакции
восстановление базы данных

Механизм

WAL базы данных

Знает про:

Его задача:

Журнал файловой системы

Файловая система не знает:

Теперь особенно важно не смешать два уровня.

44. Почему журнал ext4 не заменяет WAL PostgreSQL

```
все inode
все extents
все записи каталогов
```

Механизм

Это еще ничего не говорит о том:

Файловая система видит байты и свои метаданные.

База данных видит транзакционное состояние.

Поэтому нужны два протокола восстановления на разных уровнях.

Допустим файловая система после сбоя идеально восстановила:

45. Почему WAL PostgreSQL не заменяет журнал файловой системы

01

Обратная сторона.

02

Если файловая система после сбоя не может корректно восстановить собственные метаданные и отображение блоков, база данных не сможет заменить этот нижний уровень своим WAL.

03

Протокол восстановления СУБД строится поверх корректного контракта файловой системы и накопителя.

PostgreSQL может иметь идеальный WAL, но его файлы все равно лежат на файловой системе.

46. Где мы останавливаемся в этой лекции

```
fsync
↓
гарантированное сохранение файла
↓
WAL
↓
гарантированное сохранение транзакции
+
восстановление
↓
групповой COMMIT
```

Механизм

Здесь достаточно модели:

Здесь они только перегрузили бы лекцию про путь записи до накопителя.

Подробности конкретной реализации PostgreSQL оставляем на блок PostgreSQL:

47. Мост к будущей лекции PostgreSQL

```
SQL COMMIT
↓
БД формирует WAL
↓
дописывает WAL
↓
страничный кеш Linux / файловая система
↓
fsync / операция гарантированного сохранения
↓
блочный слой
↓
NVMe
↓
контроллер SSD
↓
энергонезависимый носитель
↓
завершение операции
↓
БД знает, что нужная часть WAL гарантированно сохранена
↓
подтверждение COMMIT
```

Механизм

А страницы данных:

Вот теперь WAL действительно вытекает из fsync(), а не появляется внезапно отдельным термином.

Теперь полный путь COMMIT выглядит так:

48. Журналирование файловой системы

```
изменение файловой системы
↓
транзакция журнала
↓
порядок гарантированного сохранения
↓
обновление метаданных
```

Механизм

Например ext4/XFS используют журналирование метаданных и протоколы порядка записи.

Упрощенно:

fsync() может заставить закрыть эту цепочку для конкретного файла.

Файловая система тоже должна корректно восстанавливаться после сбоя.

49. Зачем метаданным ФС гарантированное сохранение

```
новый блок содержит правильные данные
```

Механизм

Представим: данные блока записаны.

Но после сбоя inode не указывает на него.

Получили поврежденный файл.

Надежное сохранение включает и метаданные.

Правильных байтов недостаточно: файловая система должна уметь их найти.

50. Отложенное выделение блоков

```
write()
↓
измененные страницы кеша
↓
фоновая запись
↓
выделение экстентов
```

Механизм

Плюсы отложенного выделения:

более крупные экстенты;

меньше фрагментации;

лучше последовательная запись.

Цена - возможные поздние ошибки.

Файловая система может отложить выбор физических блоков:

51. Почему успешный `write()` не гарантирует отсутствие `ENOSPC` позже

```
ошибка может прийти позже
```

Механизм

Блоки могут быть выделены только при фоновой записи.

Поэтому успешный write() еще не исключает ENOSPC при fsync() и других последующих операциях.

Обычная запись могла лишь изменить страничный кеш, еще не выделив место на накопителе.

52. Ошибка ввода-вывода при writeback

```
write()
↓
страничный кеш
↓
успешный возврат
↓
фоновая запись
↓
ошибка устройства
```

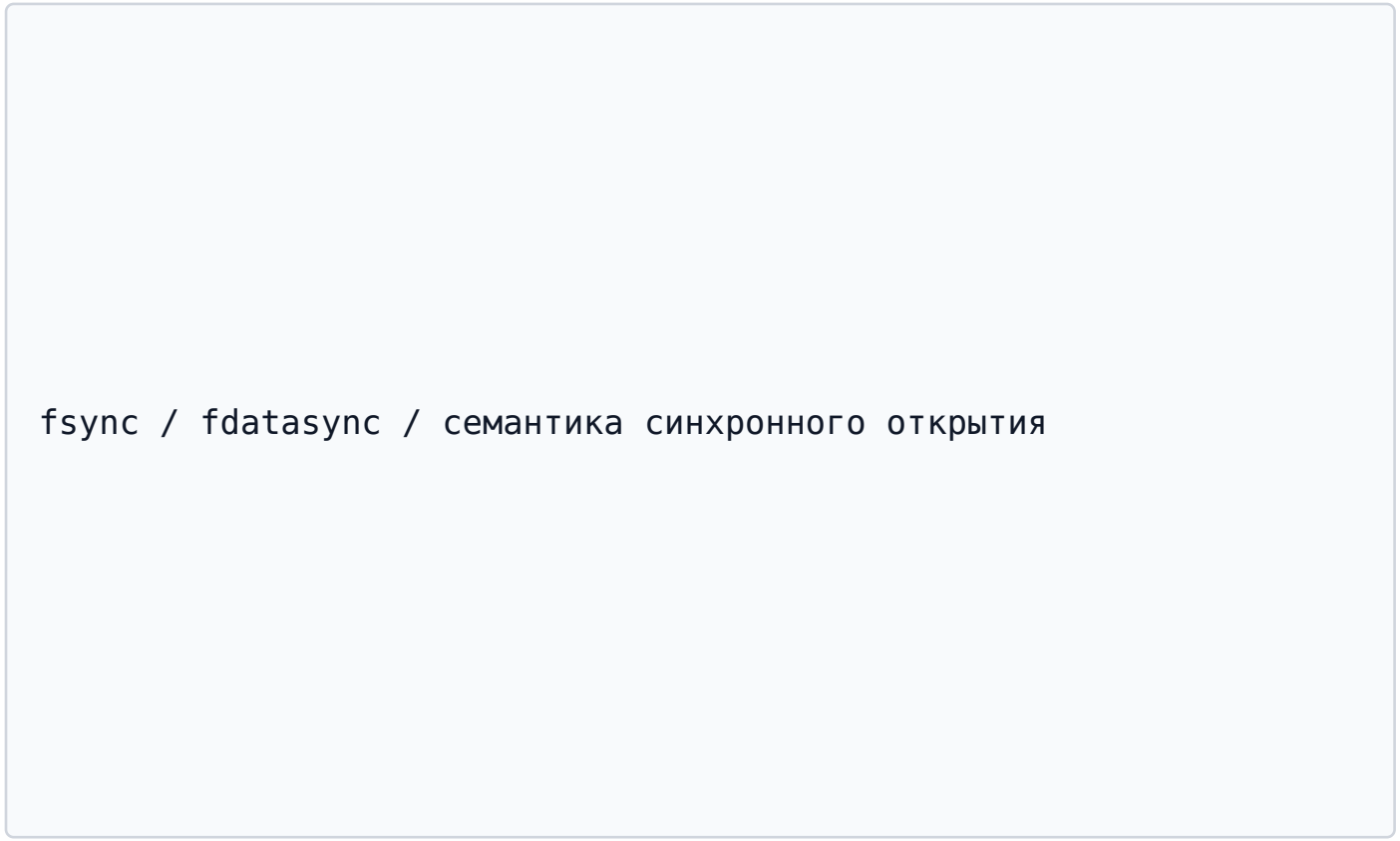
Механизм

Ошибка устройства тоже может проявиться после успешного write().

Позднюю ошибку важно учитывать при синхронизации файла.

Успешный write() еще не означает ни отсутствие поздних ошибок, ни сохранность данных.

53. `close()` не заменяет `fsync()`



Механизм

Ошибка в модели: «закрыли файл - данные надежно записаны».

close() закрывает дескриптор.

Для явных гарантий сохранения требуется fsync(), fdatasync() или режим синхронной записи.

Закроем файл — все точно запишется.

54. Долговечность каталога

```
записать временный файл
↓
fsync(file)
↓
rename
↓
fsync(directory)
```

Механизм

Шаблон, устойчивый к сбою часто включает:

Потому что:

и:

разные обязательства.

Создание/rename файла меняет метаданные каталога.

55. Атомарный `rename`

```
старое имя
↓
атомарно указывает на новое состояние inode/файла
```

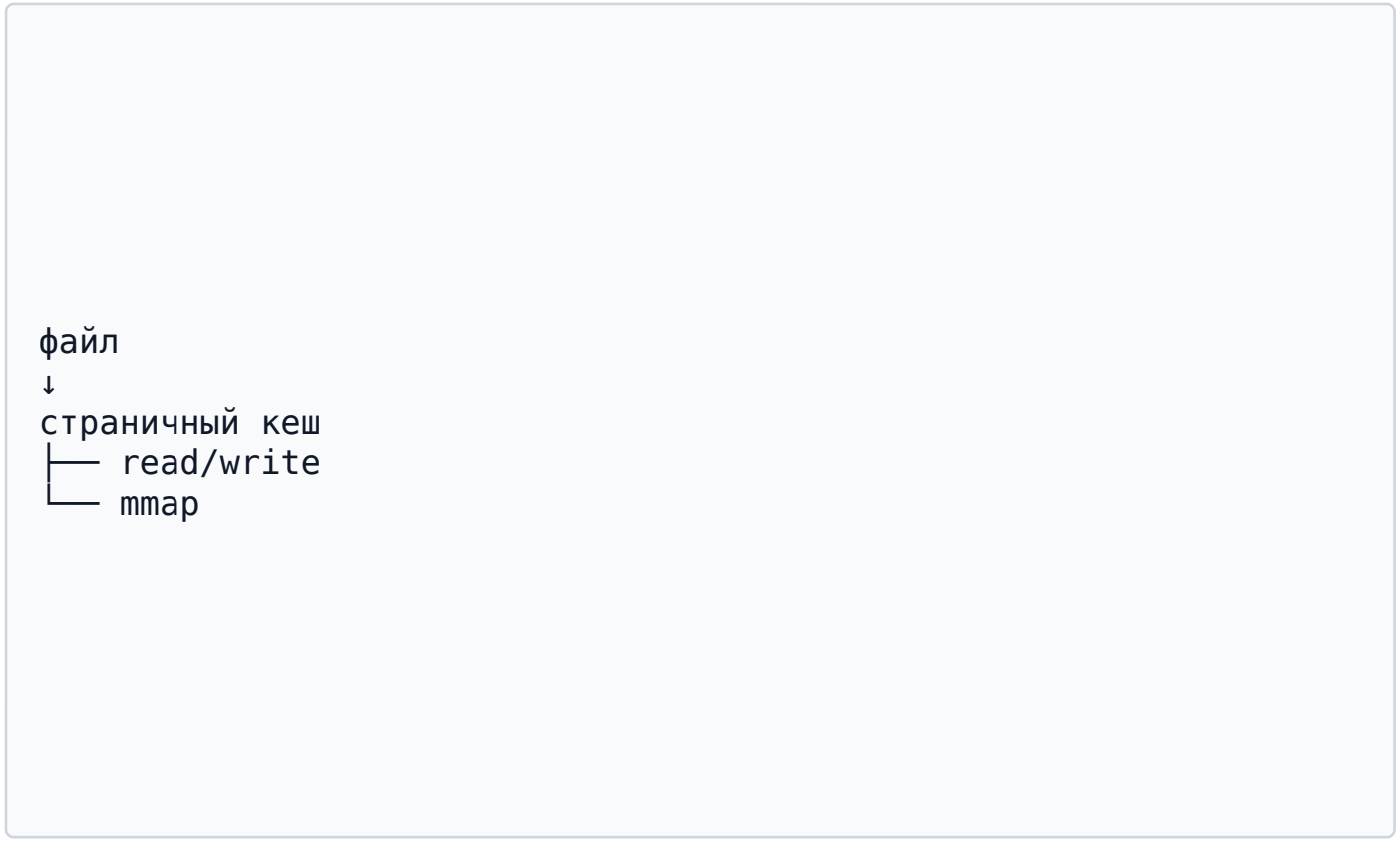
Механизм

rename() атомарно меняет видимое имя при выполнении условий ФС.

Но атомарность переименования и переживание сбоя - разные гарантии. Для второй нужен fsync каталога.

rename() обеспечивает атомарность смены имени, а синхронизация - сохранность после сбоя.

56. `mmap` файла и страничный кеш



Механизм

mmap не означает автоматический обход страничного кеша.

Файл, отображенный в память обычно использует тот же страничный кеш.

57. `msync()`

```
msync()
```

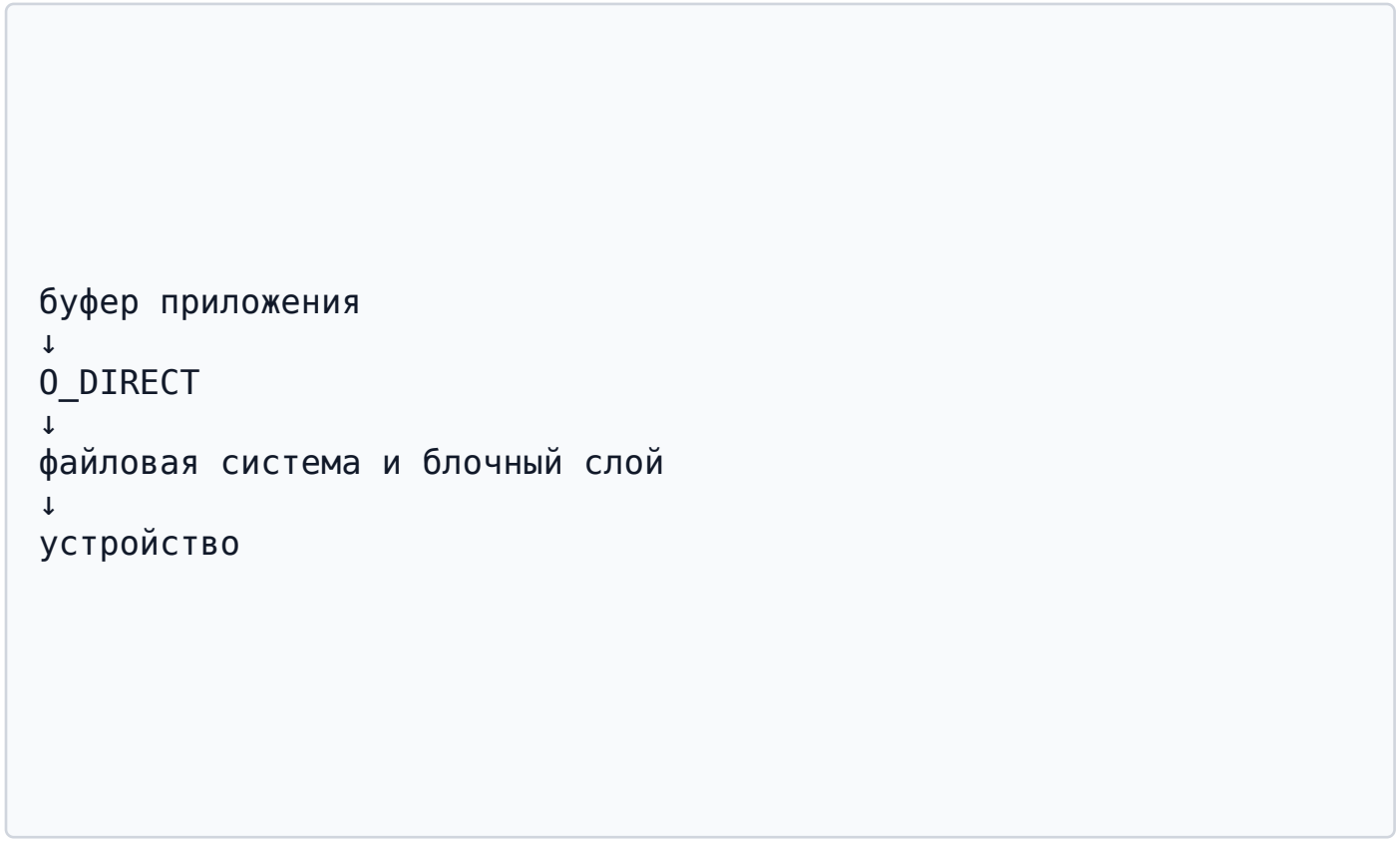
Механизм

Он управляет синхронизацией отображения с исходным файлом.

Но семантику метаданных/долговечности файловой системы все равно нужно понимать отдельно.

Для измененных страниц mmap есть:

58. Прямой ввод-вывод



Механизм

Прямой ввод-вывод позволяет обойти обычный кеш файловых данных ядра.

Но внутренние кеши SSD остаются, а запись не обязательно становится синхронной.

O_DIRECT обходит обычный страничный кеш данных, но не все уровни кеширования.

59. Зачем СУБД прямой ввод-вывод

```
буферный пул БД
+
страничный кеш
```

Механизм

СУБД может иметь собственный буферный пул.

Если ядро держит те же страницы, возникает двойное кеширование.

Прямой ввод-вывод дает БД больше контроля над этим потоком.

Движок хранения имеет собственный буферный пул.

60. Цена прямого ввода-вывода

- Появляются ограничения:
- выравнивание;
- время жизни буферов;
- правила файловой системы/устройства.
- И приложение само отвечает за работу, которую раньше делал страничный кеш.

размеры операций ввода-вывода;

61. Буферизованный и прямой ввод-вывод

Буферизованный ввод-вывод	Прямой ввод-вывод
страничный кеш	обход обычного страничного кеша данных
кеширование ядром	приложение получает больше контроля
удобно для мелких I/O	требования к выравниванию
запись может быстро завершиться	путь ближе к устройству
возможно двойное кеширование	полезно для буферных пулов БД

Ни один подход не универсально лучше.

62. Переходим в блочный слой

какие логические блоки нужно читать/писать

Механизм

Файловая система определила логические блоки для чтения или записи.

Дальше запросы проходят через общий блочный слой Linux.

Это граница между семантикой файловой системы и устройством.

63. Блочное устройство

```
смещение в файле
↓
отображение экстентов/блоков
↓
логический адрес блока
```

Механизм

Файловая система преобразует смещение файла в логический адрес блока.

NVMe/SSD представлен ядру как блочное устройство.

SSD/NVMe представлен Linux как блочное устройство.

64. `bio`

```
bio
```

Механизм

bio описывает блочный ввод-вывод:

страницы и буферы;

логические сектора/LBA;

направление операции;

флаги и ограничения.

Полезная историческая/учебная сущность блочного слоя:

65. Запрос

```
bio
↓
запрос
↓
аппаратная очередь
↓
драйвер
```

Механизм

Блочный слой объединяет операции в запросы и передает их драйверу.

Внутренние структуры зависят от версии ядра и blk-mq.

Блочный слой организует I/O в запросы, которые передаются драйверу.

66. Зачем блочный слой

- Он дает общие механизмы:
- очереди;
- объединение;
- планирование;
- учет;
- ограничения устройства;
- завершение;
- Файловая система не должен знать внутренности каждого контроллера.

справедливое распределение.

67. `blk-mq`

```
blk-mq
```

Механизм

blk-mq - многоочередной блочный слой.

Он масштабирует обработку запросов, избегая одной глобальной очереди на все CPU.

Локальные очереди снижают конкуренцию между ядрами при интенсивном вводе-выводе.

68. Знакомый паттерн рантайма

```
горячий путь локален
↓
глобальная координация реже
```

Механизм

То же было:

в планировщике Go;

в mcache;

в локальной работе GC;

в блочном слое Linux.

Снова видим:

69. Планировщик ввода-вывода

```
none
mq-deadline
```

Механизм

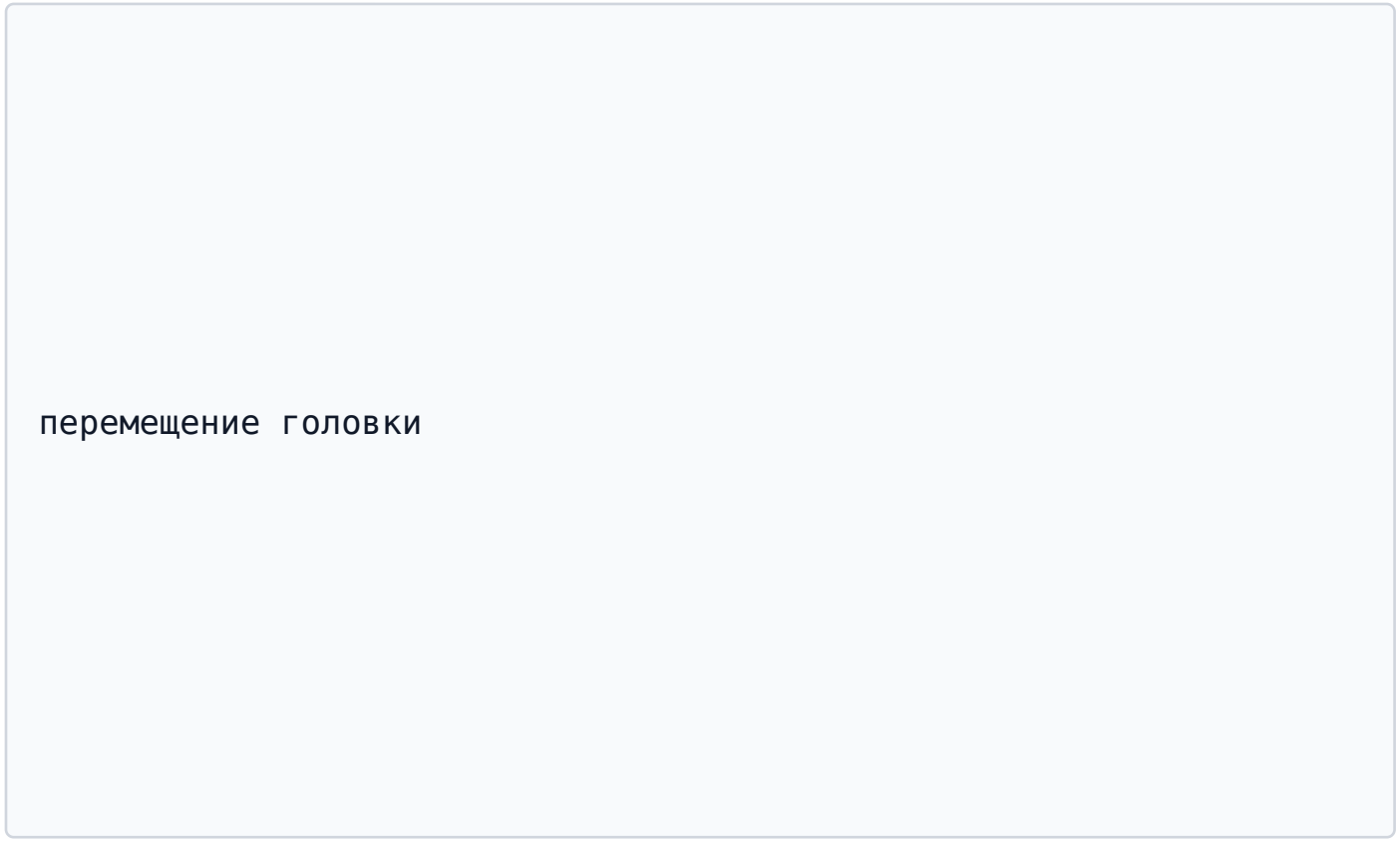
Для NVMe возможны политики none и mq-deadline.

Планировщик может объединять и переупорядочивать операции.

Выбор зависит от ядра, накопителя и нагрузки.

Перед передачей запросов устройству может работать планировщик ввода-вывода.

70. Почему NVMe часто не нужен планировщик в стиле HDD



Механизм

Для HDD дорого перемещение головки. Планировщик может экономить его переупорядочиванием.

У NVMe механики нет, зато много аппаратных очередей.

Переупорядочивание запросов может радикально экономить механику.

71. `none` не означает «очередей нет»

none

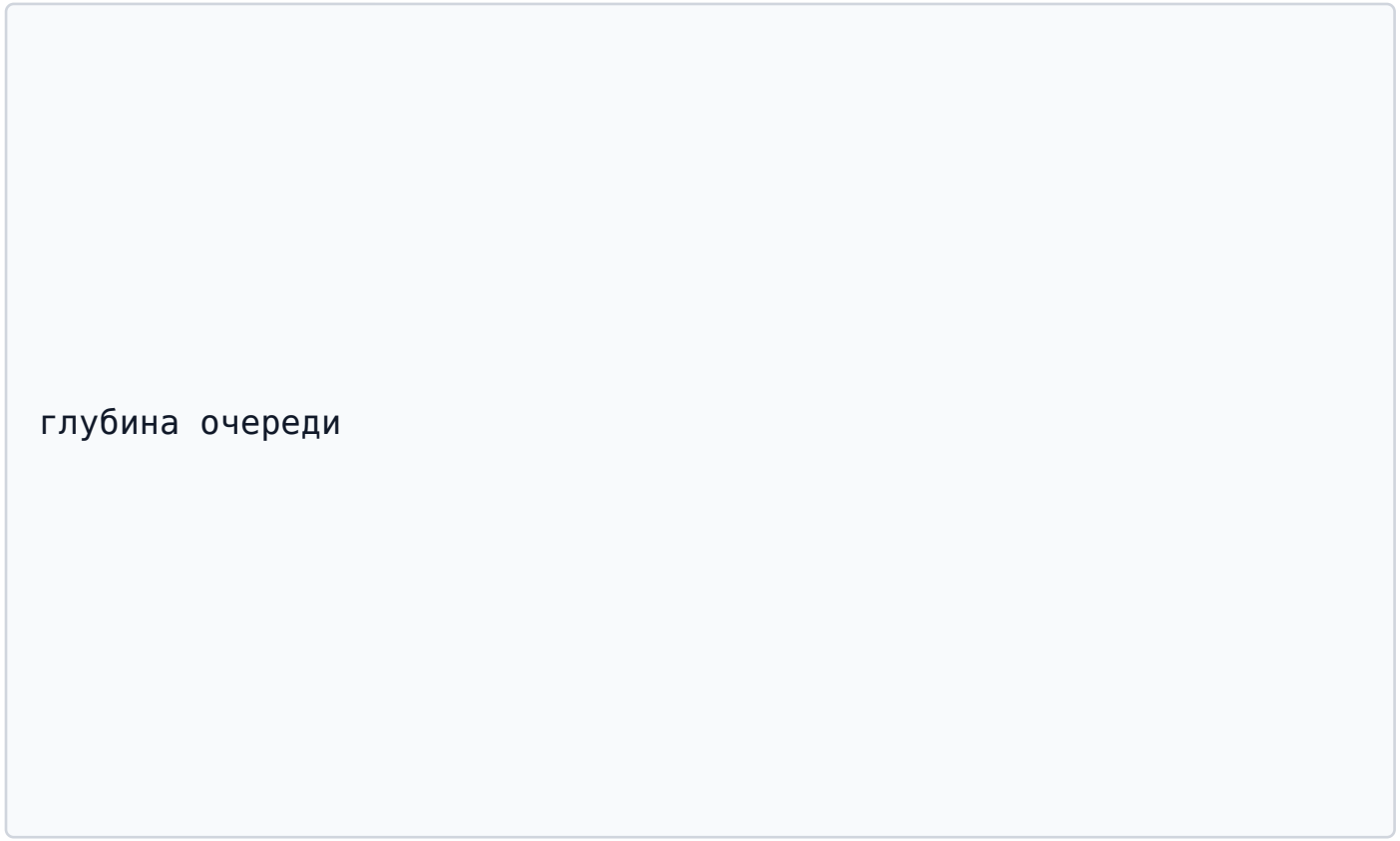
Механизм

Отсутствие отдельного планировщика не отменяет очередей.

Остаются очереди blk-mq, драйвера, NVMe и внутренняя работа NAND.

none отключает отдельный алгоритм планирования, но не очередь ввода-вывода.

72. Глубина очереди



Механизм

Глубина очереди - число запросов, одновременно находящихся в работе.

Малая глубина ограничивает загрузку устройства.

Слишком большая может увеличить задержку ожидания.

Накопитель может выполнять много I/O одновременно.

73. Закон Литтла снова работает

$$L = \lambda W$$

Механизм

Закон Литтла применим и к накопителю:

$L = \lambda W$.

Если обслуживание не ускоряется, а запросы прибывают быстрее, очередь и задержка растут.

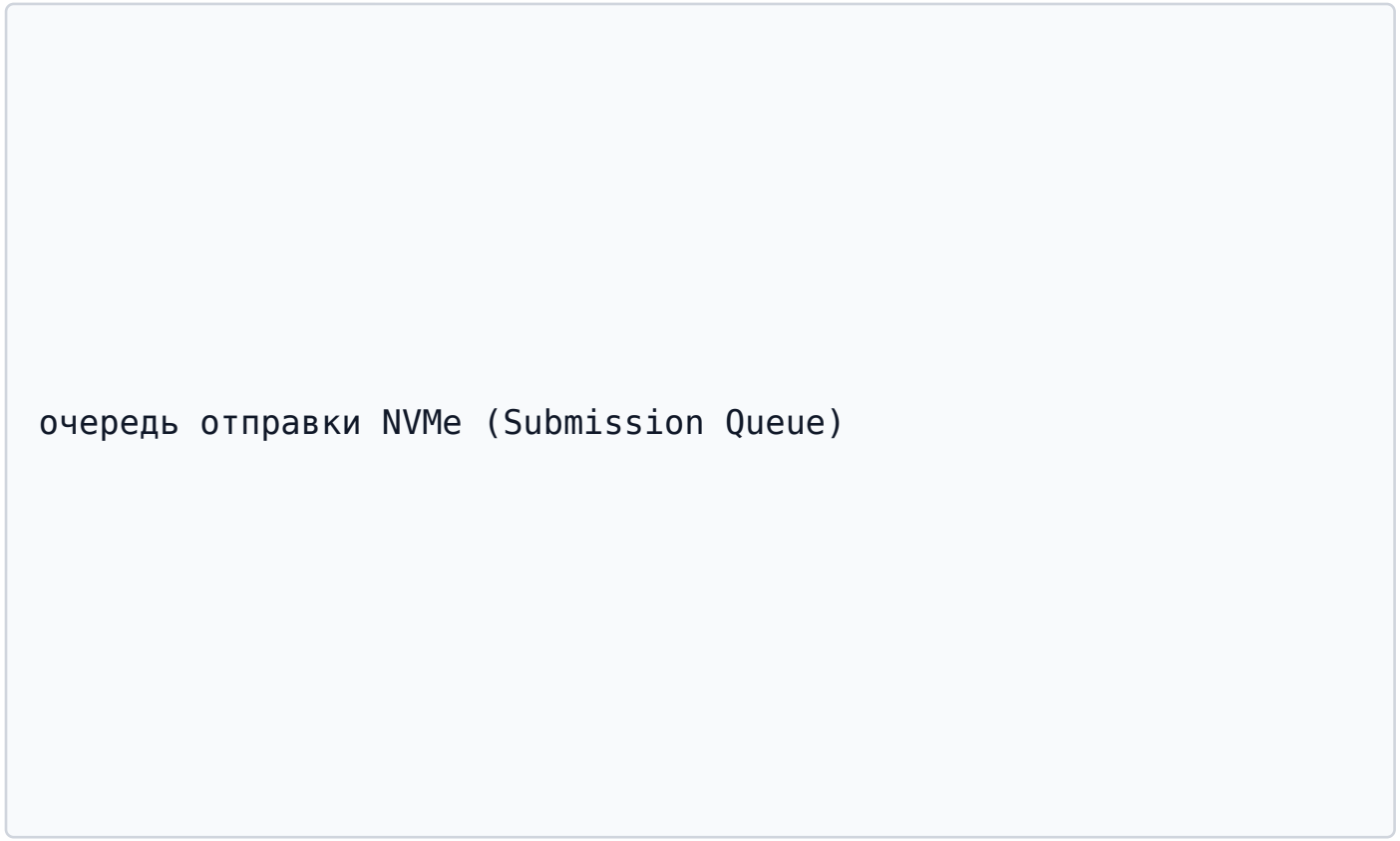
Если скорость обслуживания не растет, а очередь растет:

74. NVMe

- Протокол проектировался под быстрые флеш-устройства на PCIe.
- Ключевые свойства:
- много очередей;
- большая глубина очередей;
- низкие накладные расходы протокола;
- модель отправки/завершения;
- высокий параллелизм.

NVMe — Non-Volatile Memory Express, протокол для энергонезависимых накопителей.

75. Очередь отправки (Submission Queue)



Механизм

Упрощенно команда содержит:

код операции;

пространство имен;

LBA;

длина;

Драйвер передает команды контроллеру через очередь отправки.

76. Регистр doorbell

```
регистр doorbell
```

Механизм

Драйвер обновляет:

В очередь отправки появились новые команды.

Дальше контроллер сам забирает работу.

Обновление doorbell сообщает контроллеру о новых командах в очереди.

77. Очередь завершений (Completion Queue)

```
контроллер NVMe
↓
очередь завершений (Completion Queue)
```

Механизм

После завершения:

прерывание;

опрос;

гибридные механизмы.

Потом завершение поднимается вверх по стеку.

Прерывание или опрос завершений сообщает ОС о готовности операции.

78. Полный цикл NVMe

```
блочный запрос
↓
очередь отправки NVMe
↓
регистр doorbell
↓
контроллер
↓
операция с флеш-памятью
↓
очередь завершений
↓
прерывание или опрос
↓
драйвер
↓
завершение запроса
↓
файловая система
↓
пробуждение ожидающей задачи
```

Вот что может скрываться под одним fsync().

79. Прерывания и опрос

```
завершение
↓
прерывание
↓
обработчик
↓
разбудить задачу
```

Механизм

Прерывание не требует постоянно проверять устройство.

Активный опрос тратит CPU, зато в некоторых режимах снижает задержку завершения.

Активный опрос меняет загрузку CPU на меньшую задержку.

80. Контроллер SSD

- Контроллер SSD выполняет:
- планирование команд;
- трансляцию FTL;
- сборку мусора;
- выравнивание износа;
- коррекцию ошибок ECC;
- учет дефектных блоков;
- управление параллелизмом.

Под протоколом NVMe находится сам контроллер SSD.

81. NAND не является RAM

```
записать по адресу
```

Механизм

DRAM допускает обычную перезапись ячейки.

В NAND программирование устроено иначе: для повторного использования нужен сброс более крупного блока.

NAND нельзя рассматривать как энергонезависимую RAM с обычной перезаписью.

82. FTL

```
логический адрес блока
↓
физическое расположение в NAND
```

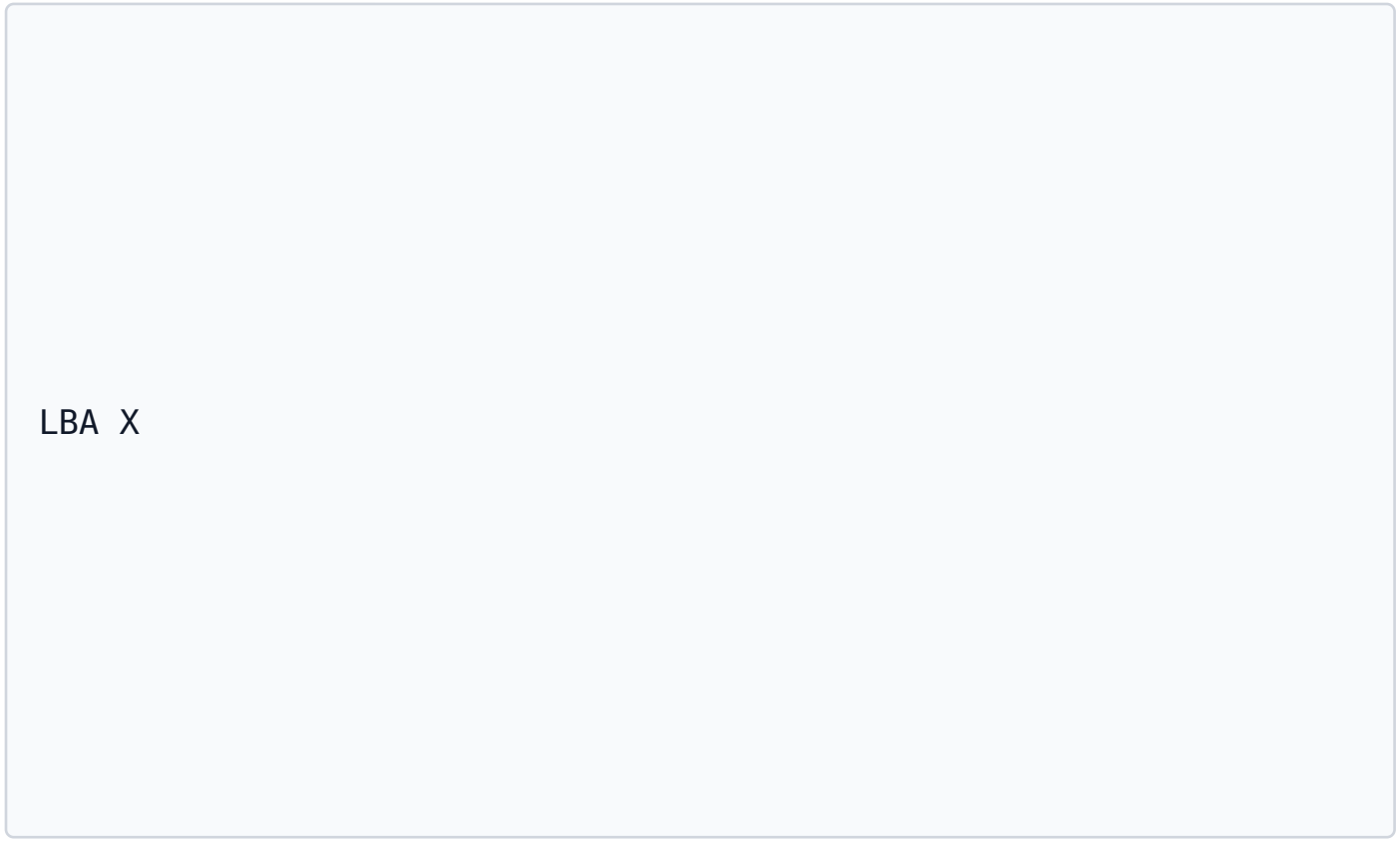
Механизм

FTL сопоставляет логические адреса блоков с физическими страницами NAND.

Контроллер сам выбирает расположение и перемещает данные.

FTL — Flash Translation Layer, слой трансляции флеш-адресов.

83. Запись в новое место (`out-of-place update`)



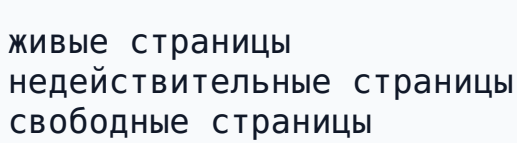
Механизм

Хост обращается к LBA X.

Контроллер может записать новую версию данных в другое место NAND и обновить FTL.

Логический адрес не обязан совпадать с постоянным физическим расположением в NAND.

84. Сборка мусора внутри SSD



Механизм

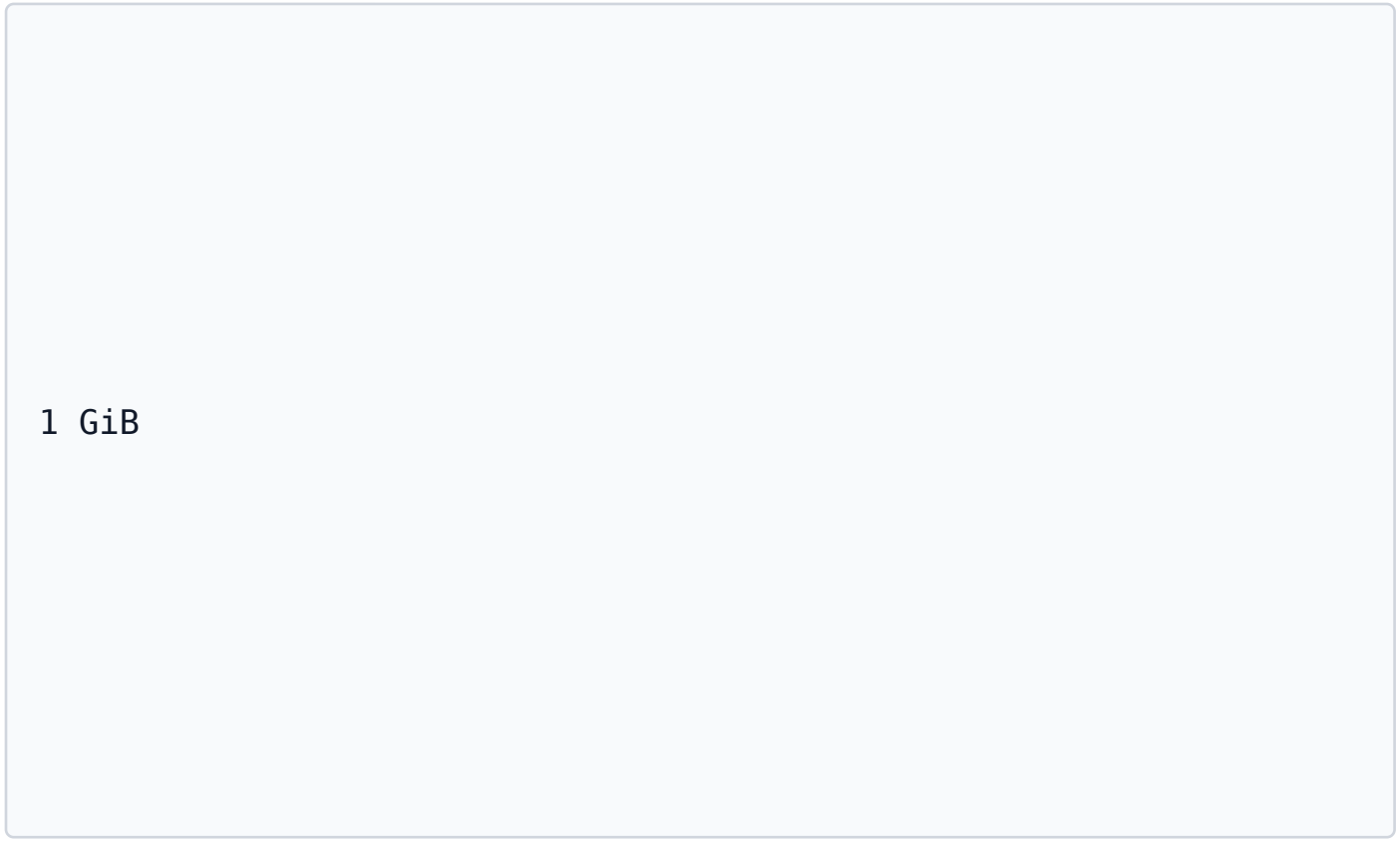
Блок стирания содержит живые и недействительные страницы.

Чтобы освободить его, контроллер переносит живые данные и стирает блок.

Это сборка мусора SSD, не Go GC.

Это внутренняя сборка мусора SSD.

85. Усиление записи (`write amplification`)



Механизм

Хост записал 1 GiB, но NAND может записать больше из-за:

переноса данных;

сборки мусора;

служебных метаданных.

Усиление записи - отношение физических записей NAND к объему записи хоста.

86. Почему усиление записи важно

```
внутренняя работа ↑
↓
задержка ↑
↓
эффективная пропускная способность ↓
↓
износ NAND ↑
```

Механизм

Чем он выше:

Накопитель может выглядеть медленным не потому, что хост отправляет много данных, а потому что контроллер делает много внутренней работы.

87. Резервирование дополнительной емкости (`over-provisioning`)

```
GC проще
↓
усиление записи ниже
↓
длительные записи стабильнее
```

Механизм

Резервные блоки нужны для:

сборки мусора;

выравнивания износа;

замены дефектных блоков;

поддержания свободного запаса.

SSD обычно имеет большую емкость NAND, чем отдает пользователю.

88. Почему почти заполненный SSD может резко деградировать

```
новая запись
↓
нужна сборка мусора SSD
↓
перенос живых страниц
↓
стирание блока
↓
запись
```

Механизм

То есть одна запись хоста может вызвать много фоновой работы.

Поэтому:

и:

могут давать очень разную длительную задержку.

Когда свободных блоков мало:

89. SLC-кеш

```
короткий всплеск записи
↓
SLC-кеш
↓
очень высокая пропускная способность
```

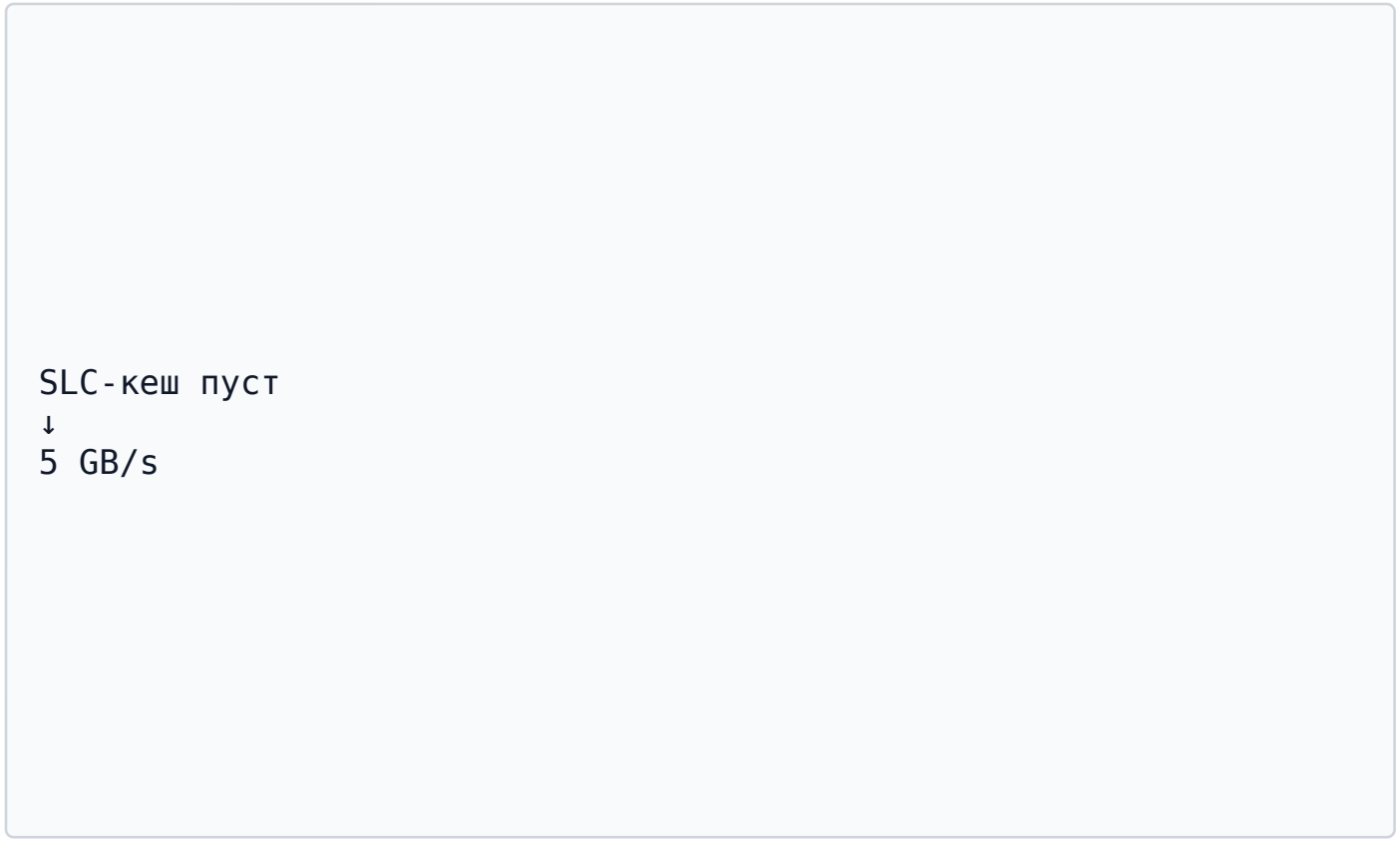
Механизм

Учебно:

После заполнения:

Многие потребительские SSD используют часть NAND как быстрый псевдо-SLC-кеш.

90. Почему короткий тест производительности врет



Механизм

Первые секунды накопитель пишет в быстрый SLC-кеш.

После его заполнения скорость может резко упасть.

Короткий тест не характеризует устойчивую запись.

Для длительной нагрузки нужно измерять скорость после исчерпания быстрого кеша.

91. DRAM-кеш устройства

- Некоторые SSD имеют DRAM.
- метаданных FTL;
- кеш отображений;
- состояния контроллера;
- буферизации.
- Но возникает вопрос долговечности:
- Если контроллер подтвердил команду, данные уже гарантированно сохранены?
- Ответ зависит от семантики устройства.

Она может использоваться для:

92. Энергозависимый кеш записи

```
потеря питания
↓
данные потеряны
```

Механизм

Если накопитель подтвердил запись до сохранения ее в энергонезависимой памяти, потеря питания опасна.

Хост должен требовать нужных гарантий через Flush/FUA.

Если устройство признало запись завершенной, но данные еще только в энергозависимом кеше:

93. Flush: сброс кеша накопителя

```
fsync
↓
порядок записи файловой системы
↓
блочный слой
↓
сброс устройства
↓
контроллер сбрасывает энергозависимое состояние
↓
граница гарантированного сохранения
```

Механизм

Flush - команда сброса накопленных записей.

Она связывает fsync() с гарантиями самого устройства.

Сделать ранее подтвержденные записи гарантированно сохраненными согласно контракту устройства.

94. FUA

```
конкретная запись
↓
завершение только после выполнения нужной семантики
гарантированного сохранения
```

Механизм

FUA - Force Unit Access.

Для конкретной записи устройство должно выполнить оговоренные гарантии сохранения до подтверждения ее завершения.

Файловая система и блочный слой могут использовать FUA и сброс в комбинации.

95. Flush и FUA — разные операции

```
сброс
→ гарантированно сохранить накопленные ранее записи

FUA
→ конкретная запись должна считаться гарантированно сохраненной
после завершения
```

Механизм

Flush относится к накопленным записям; FUA задает семантику конкретной операции.

Точный выбор зависит от файловой системы и протокола накопителя.

Реальная последовательность зависит от протокола, устройства и файловой системы.

96. Защита от потери питания

PLP — Защита от потери питания

Механизм

Например конденсаторы дают контроллеру время сохранить критичное энергозависимое состояние.

Это особенно важно для:

баз данных;

WAL;

метаданных;

Защита от потери питания помогает контроллеру завершить критические операции.

97. Почему потребительские и корпоративные SSD — не одна категория

- Потребительский SSD может иметь:
- небольшой SLC-кеш;
- сильный разброс задержки;
- ограниченную PLP;
- агрессивную сборку мусора;
- оптимизацию под короткие нагрузки.

- Корпоративные SSD рассчитаны на длительный поток записи и надежность.

База данных полагается на контракт гарантированного сохранения накопителя.

98. TRIM / discard

эти логические блоки больше не нужны

Механизм

Файловая система знает, какие логические блоки больше не используются.

TRIM/discard передает эту информацию SSD для сборки мусора и учета свободного места.

контроллер SSD этого сам не знает.

99. Онлайн discard и `fstrim`

```
discard сразу
```

Механизм

Освобожденные блоки можно сообщать накопителю сразу либо периодически через fstrim.

Выбор зависит от файловой системы и нагрузки.

Конкретная политика зависит от файловой системы и устройства.

100. Полная цепочка `write()`

```
Go `f.Write()`
↓
системный вызов `write()`
↓
VFS
↓
файловая система
↓
страничный кеш
↓
скопировать данные
↓
пометить страницу измененной
↓
учет измененных страниц
↓
`запись()` возвращается
```

До гарантированного сохранения на накопителе еще далеко.

101. Полная цепочка фоновой записи

```
измененные страницы накапливаются
↓
запускается writeback
↓
файловая система отображает блоки/экстенты
↓
bio / запросы
↓
blk-mq
↓
очереди NVMe
↓
контроллер SSD
↓
NAND
↓
завершение
↓
страница чистая
```

Смотрим на механизм, цену и симптом под нагрузкой.

102. Полная цепочка `fsync()`

```
приложение вызывает `fsync`
↓
записать измененные данные файла
↓
записать необходимые метаданные / состояние журнала
↓
отправить блочный ввод-вывод
↓
дождаться завершений
↓
при необходимости выполнить сброс/FUA
↓
устройство подтверждает требуемую семантику сохранения
↓
`fsync` возвращается
```

Это основной путь гарантированного сохранения.

103. Что будет при 50 000 мелких записей/с

```
50 тыс. записей/с
↓
страничный кеш поглощает всплеск
↓
Dirty ↑
↓
фоновый writeback
```

Механизм

Если накопитель успевает за потоком, фоновая запись очищает страницы.

Если нет, Dirty растет и ядро замедляет писателей.

Буферизованная запись скрывает задержку до насыщения очереди Dirty.

104. Что будет при 50 000 `fsync()`/s

```
частота `fsync`
>
пропускная способность устройства по сбросу
```

Механизм

Если частота fsync() превышает возможности накопителя по сбросу, растут очереди и задержка.

Страничный кеш не скрывает цену гарантированного сохранения.

Каждая синхронная операция должна дождаться гарантий сохранения.

105. Почему пакетирование критично

```
1 запись
↓
fsync
```

Механизм

Было:

Стало:

Это один из фундаментальных механизмов высокой производительности WAL.

Цена гарантированного сохранения амортизируется.

106. Хвост задержки `fsync()`

```
p50 низкий
p99 намного выше
p99.9 иногда очень высокий
```

Механизм

Причины скачков p99 fsync():

сборка мусора SSD;

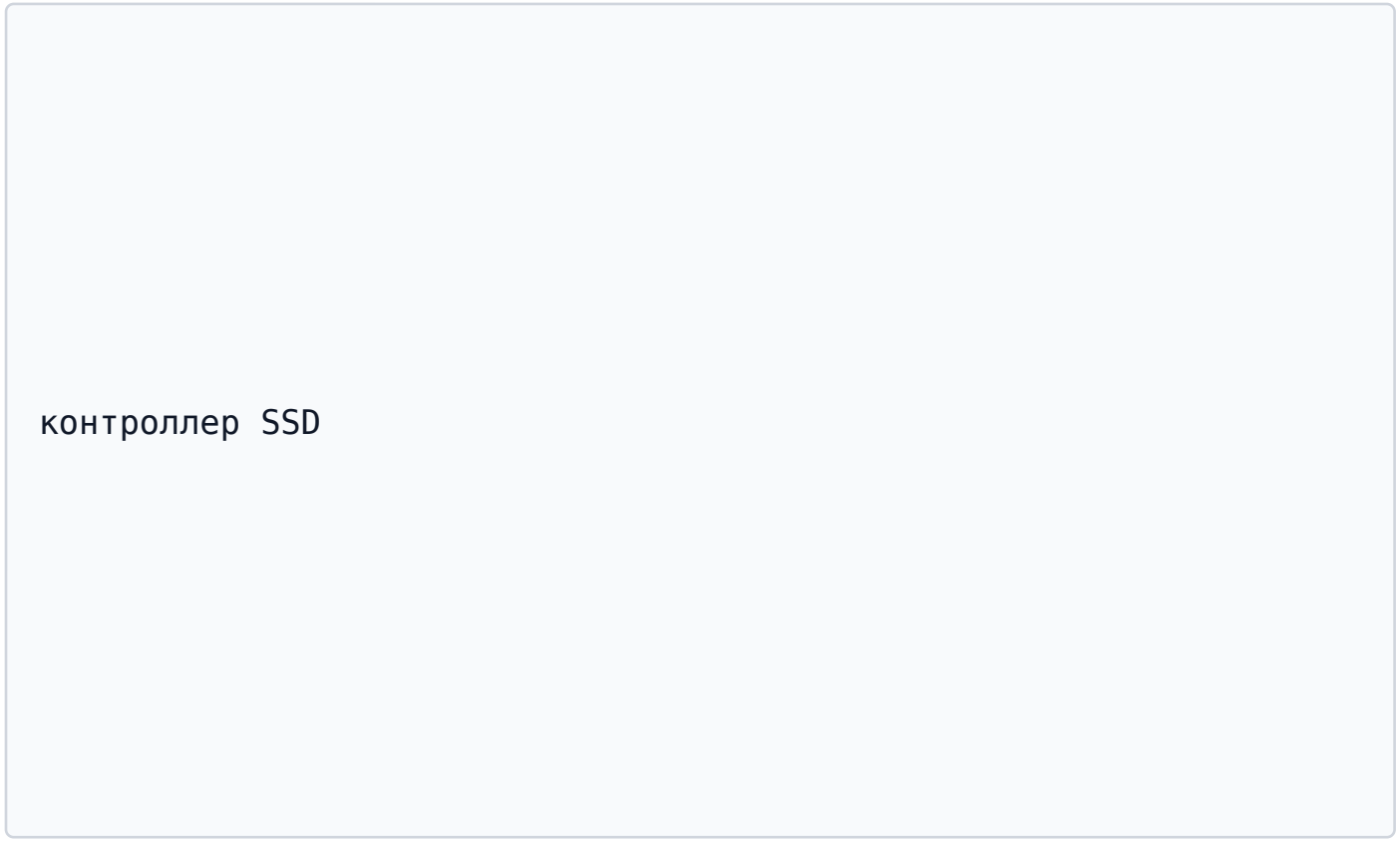
выравнивание износа;

перенос данных из SLC-кеша;

работа с метаданными FTL.

Даже при низкой средней задержке устройство может периодически выполнять тяжелые операции.

107. Очереди внутри устройства



Механизм

Контроллер SSD может иметь собственную внутреннюю очередь.

Поэтому нужно отличать ожидание в Linux от очередей внутри устройства.

Даже если очереди Linux короткие:

108. IOPS — не вся история

```
1 000 000 IOPS
```

Механизм

Рекорд IOPS в спецификации может быть получен при QD64, случайном чтении 4 KiB и определенном сочетании операций.

Это не гарантирует ту же скорость для WAL и fsync().

Показатели IOPS всегда привязаны к условиям измерения.

109. Последовательная пропускная способность — не задержка WAL

```
7 GB/s последовательной записи
```

Механизм

7 GB/s последовательной записи ничего не говорит о задержке отдельного fsync().

Пропускная способность и задержка - разные характеристики.

Скорость потока данных не гарантирует низкую задержку отдельного сброса.

110. Случайная запись

- разбросанные отображения;
- недействительные страницы;
- перемещение данных;
- внутренняя сборка мусора.
- Поэтому усиление записи часто выше.

Случайные записи усложняют FTL:

111. Последовательная запись

```
WAL: допись в конец
```

Механизм

Последовательный поток проще объединять и размещать на накопителе.

Поэтому допись WAL часто выгоднее множества случайных обновлений.

Последовательные записи хоста обычно удобнее для размещения и объединения.

112. Страничный кеш и буферный пул БД

```
кеш БД
+
страничный кеш
```

Механизм

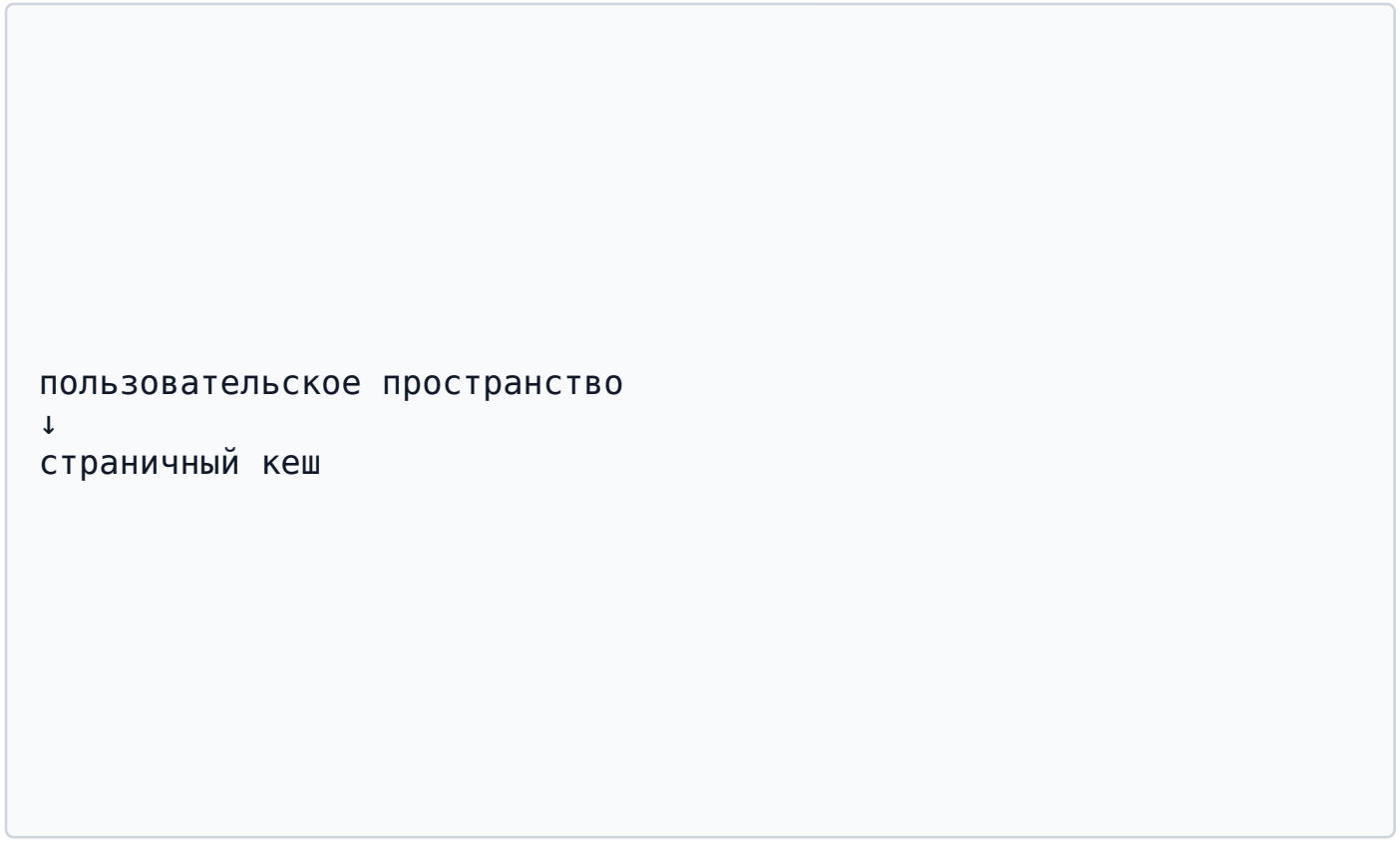
СУБД может держать страницы в собственном буферном пуле.

Дополнительный страничный кеш ядра иногда приводит к двойному кешированию.

Прямой ввод-вывод дает БД больше контроля.

БД может иметь собственный буферный пул.

113. WAL и страничный кеш



Механизм

WAL тоже может сначала попасть в страничный кеш.

Но для подтверждения COMMIT требуется гарантированно сохранить нужную позицию журнала.

Кеш ускоряет запись WAL, а синхронизация определяет момент надежного COMMIT.

114. Путь COMMIT в стиле PostgreSQL

```
транзакция
↓
сформировать запись WAL
↓
дописать WAL
↓
выполнить сбросить WAL
↓
COMMIT подтвержден
```

Механизм

Учебно:

Долговечность переносится на журнал.

Страницы данных могут быть записаны позже, после сохранения необходимого WAL.

115. Что если гарантированное сохранение отключить

данные могут существовать только в энергозависимых слоях

Механизм

Если подтверждать операцию до сохранения данных в энергонезависимой памяти, внезапная потеря питания может привести к потере подтвержденных записей.

Это изменение гарантий корректности.

Отключение гарантированного сохранения меняет контракт, а не только скорость.

116. Подтверждение устройства еще нужно правильно понимать

```
команда завершена согласно своей семантике
```

Механизм

Подтверждение команды означает ее завершение по правилам устройства.

Гарантии сохранности зависят от кеша записи, Flush/FUA и защиты от потери питания.

Но гарантированное сохранение зависит от:

117. Почему UPS не заменяет семантику гарантированного сохранения накопителя

- Но остаются:
- panic ядра;
- сброс контроллера;
- PCIe ошибка;
- сбой прошивки;
- перезагрузка хоста;
- сбой цепи питания.
- Корректность строят на документированном протоколе гарантированного сохранения.

UPS снижает риск полного потеря питания.

118. Диагностика: `iostat`

```
iostat -x 1
```

Что измеряем

Для общей картины дискового ввода-вывода используем iostat -x 1.

Смотрим IOPS, пропускную способность, размеры запросов и задержку.

Состав колонок зависит от версии sysstat.

Названия колонок могут различаться между версиями sysstat.

119. `await`

```
await
```

Механизм

await - средняя задержка операций ввода-вывода, наблюдаемая со стороны хоста.

Она включает ожидание в доступных для измерения очередях.

Это агрегированная метрика, не latency NAND.

await отражает среднюю задержку операций, наблюдаемую со стороны Linux.

120. `%util`

100%

Механизм

На старом одноочередном устройстве %util=100% часто означал насыщение.

У многоочередного NVMe этого недостаточно.

Сопоставляйте задержку, глубину очереди, IOPS и пропускную способность.

%util нельзя трактовать как процент использованной мощности NVMe.

121. `pidstat -d`

```
pidstat -d 1
```

Что измеряем

pidstat -d 1 помогает связать ввод-вывод с конкретным процессом.

Но фоновую запись страниц может выполнять ядро позднее.

Активность процесса и фактическая запись на накопитель могут расходиться по времени.

122. `/proc/<pid>/io`

```
байты, переданные через системные вызовы
```

Механизм

В /proc/<pid>/io видны счетчики ввода-вывода процесса.

Нужно отличать логические bytes системных вызовов от физического I/O.

Страничный кеш разносит эти события во времени.

Показатели записи процесса и устройства измеряют разные этапы.

123. `/proc/meminfo`

```
Dirty
Writeback
Cached
```

Механизм

В /proc/meminfo полезны Dirty, Writeback и Cached.

Если Dirty растет устойчиво, очередь записи увеличивается.

Writeback показывает страницы, уже находящиеся в записи.

Dirty и Writeback помогают оценить накопление и сброс измененных страниц.

124. `vmstat`

- заблокированные задачи;
- ожидание ввода-вывода;
- swapping;
- системную нагрузку.
- Это общий индикатор.
- Не профилировщик накопителя.

На границе память/накопитель полезно смотреть:

125. `iotop`

кто сейчас создает/получает ввод-вывод?

Что измеряем

iotop помогает найти процессы, связанные с дисковым вводом-выводом.

Но из-за страничного кеша приложение может создавать изменения сейчас, а диск записывать их позже.

Но страничный кеш может размывать причинность:

126. `strace`

```
strace -ttT -e trace=write,fsync,fdatasync ./app
```

Что измеряем

Запустить strace с измерением длительности write(), fsync() и fdatasync().

Сопоставить системные вызовы с задержкой приложения.

strace показывает, где приложение реально ждет на границе системного вызова.

127. `blktrace`

```
очередь
передача устройству
завершение
```

Что измеряем

blktrace позволяет наблюдать постановку запросов в очередь, отправку устройству и завершения.

Это уже уровень блочного слоя, а не файлового API.

Глубокий уровень блочный слой.

128. eBPF / bpftrace

```
фоновая запись
блочные запросы
fsync
завершение ввода-вывода
```

Что измеряем

И строить гистограммы задержки.

Плюс:

Минус:

Нужно точно понимать, какой слой измеряется.

Можно использовать tracepoint'ы блочного слоя/файловой системы:

129. `perf`

- memcpy;
- код файловой системы;
- контрольные суммы;
- прерывания/опрос;
- завершение блочной операции.
- Но ожидание ввода-вывода может вообще не проявляться как горячий стек CPU.

perf показывает стоимость пути записи по CPU:

130. Почему `write()` может быть медленным без `fsync`

- Причины:
- выделение страниц;
- давление на память;
- блокировки ФС;
- отложенное выделение блоков;
- квоты и лимиты Dirty;
- перегрузка фоновой записи;
- ограничение ввода-вывода cgroup.

замедление по измененная-лимитам;

131. Контроллер ввода-вывода cgroup v2

```
io.max
io.weight
```

Механизм

Контроллер ввода-вывода cgroup v2 поддерживает io.max и io.weight.

Контейнер может упереться в лимит еще до насыщения физического накопителя.

Для контейнера граница I/O может находиться выше самого SSD.

132. Файловая система контейнера

```
приложение
↓
overlay-файловая система
↓
верхний/нижний слои
↓
файловая система хоста
↓
блочный слой
↓
устройство
```

Механизм

В контейнере запись может проходить через overlayfs и файловую систему хоста.

Поэтому тест внутри контейнера не обязательно совпадает с прямой записью в ext4/XFS.

В container путь может быть:

133. Copy-up в overlayfs

```
копирование вверх (copy-up)
↓
верхний слой
↓
изменение
```

Механизм

Файл находится в нижнем слое образа.

При первой записи overlayfs может скопировать его в верхний слой.

Для данных БД обычно используют отдельный том.

Первая запись в файл из нижнего слоя может потребовать дополнительного копирования.

134. Сетевой накопитель

```
блочный слой
↓
сеть
↓
удаленный накопитель
```

Механизм

Для EBS, Ceph или SAN путь записи включает сетевой протокол и удаленное хранилище.

Задержка fsync() тогда зависит и от сети, и от удаленного сервиса хранения.

Для сетевого тома гарантированное сохранение зависит от удаленной системы.

135. Нормальная буферизованная запись

```
скорость создания измененных страниц
<
устойчивая скорость фоновой записи
```

Механизм

Получаем:

Страничный кеш успешно сглаживает всплеск.

136. Давление на writeback

```
скорость изменения данных > скорость SSD
↓
Dirty растет
↓
фоновая запись перегружена
↓
ядро замедляет писателей
↓
задержка write() растет
↓
p99 увеличивается
```

Смотрим на механизм, цену и симптом под нагрузкой.

137. `fsync()` узкое место

```
объем записи невелик
↓
`fsync` вызывается часто
↓
доминирует задержка гарантированного сохранения на устройстве
↓
p99 транзакций ↑
```

Механизм

Частые fsync() могут ограничивать TPS даже при малом объеме данных.

Узкое место - задержка гарантированного сохранения, а не MB/s.

Малый поток данных не исключает дорогого fsync() и высокого p99.

138. Внутреннее давление SSD

```
задержка устройства скачет
↓
пропускная способность проседает
```

Механизм

Даже при неизменной нагрузке хоста SSD может периодически:

исчерпать SLC-кеш;

запустить сборку мусора;

остаться с малым запасом свободных блоков.

Источник скачков задержки может находиться внутри контроллера SSD.

139. Иллюзия страничного кеша

```
запись 10 GiB
↓
завершено за 2 с
```

Механизм

Приложение записало 10 GiB за две секунды.

Но данные могли оказаться лишь в страничном кеше, а накопитель еще пишет.

Нужен тест с правильной синхронизацией.

Быстрый write() может измерять скорость RAM, а не устойчивую скорость SSD.

140. Почему `dd` легко меряет не то

```
копирование в RAM
+
страничный кеш
+
фоновая запись
```

Механизм

Тест накопителя без понимания кеширование легко становится тестом RAM.

Обычный dd без семантики синхронного/прямого ввода-вывода может мерить смесь:

141. Почему `fio` полезнее

- fio позволяет задать:
- обычный или прямой ввод-вывод;
- размер блока;
- глубину очереди;
- режим синхронизации;
- движок ввода-вывода;
- длительность и рабочий набор;
- соотношение чтения и записи.

соотношение чтения/записи;

142. Практика 1: `write()` против `fsync()`

```
package main

import (
            "os"
)

func main() {
            f, err := os.Create("data.bin")
            if err != nil {
                        panic(err)
            }
            defer f.Close()

            buf := make([]byte, 4096)

            for i := 0; i < 100_000; i++ {
                        if _, err := f.Write(buf); err != nil {
                                    panic(err)
                        }
            }
}
```

Что делаем

Написать программу, которая делает 100 000 записей по 4 KiB.

Измерить время.

Добавить f.Sync() после каждой записи.

Измерить еще раз.

Сравнить время обычной записи и записи с гарантированным сохранением.

143. Практика 2: пакетная синхронизация

```
1 запись
↓
fsync
```

Что делаем

Сравнить два режима:

A: запись → fsync() каждый раз;

B: несколько записей → один fsync().

Измерять записи/с и число fsync()/с.

Пакетирование распределяет стоимость одного сброса между несколькими записями.

144. Практика 3: очередь измененных страниц

```
watch -n 0.5 'grep -E "Dirty|Writeback" /proc/meminfo'
```

Что делаем

Записывать данные быстрее накопителя.

Одновременно наблюдать Dirty и Writeback в /proc/meminfo.

Измерять длительность write() и искать начало замедления.

Растущая очередь Dirty приводит к принудительному замедлению писателей.

145. Практика 4: ловушка теста производительности страничного кеша

- Записать файл размером меньше RAM.
- Потом:
- fsync;
- прямой ввод-вывод через fio;
- длительный тест длиннее нескольких секунд.
- Сравнить.
- Цель:
- Понять, где заканчивается тест производительности RAM/страничный кеш и начинается тест производительности накопитель.

Сначала обычный буферизованный ввод-вывод.

146. Практика 5: `strace` путь гарантированного сохранения

```
strace -ttT -e trace=write,fsync,fdatasync ./app
```

Что делаем

Запустить strace с измерением времени write(), fsync() и fdatasync().

Сопоставить длительность системных вызовов с задержкой программы.

strace показывает время ожидания на границе приложения и ядра.

147. Практика 6: `iostat`

```
iostat -x 1
```

Что делаем

Параллельно запустить iostat -x 1.

Смотреть задержку и поток операций.

Сопоставить их с измерениями Go и strace.

Задержка системного вызова и задержка операции, видимая блочному слою, различаются.

148. Практика 7: глубина очереди

```
iodepth=1
iodepth=4
iodepth=16
iodepth=64
```

Что делаем

Через fio менять глубину очереди: 1, 4, 16, 64.

Снимать IOPS, пропускную способность и задержку.

Проверить компромисс между параллелизмом и p99.

Большая глубина очереди не обязана уменьшать задержку отдельного запроса.

149. Практика 8: краткий всплеск и длительная запись

```
30 секунд
```

Что делаем

Сравнить краткий и длительный тесты одной и той же записи.

Проверить, насколько скорость падает после исчерпания SLC-кеша.

Длительную производительность нельзя выводить из первых секунд теста.

150. Практика 9: заполнение SSD

SSD с большим запасом свободного места

Что делаем

Сравнить SSD с большим запасом свободного места и почти заполненный.

Подать одинаковую длительную случайную запись.

Следить за p95/p99 и пропускной способностью.

Эксперимент с заполнением выполнять только на отдельном тестовом накопителе.

151. Практика 10: буферизованный и прямой ввод-вывод

буферизованный

Что делаем

В fio задать два режима: обычный и прямой ввод-вывод.

Сравнить задержку, пропускную способность и нагрузку CPU.

Убедиться, что страничный кеш меняет путь операции.

Понять, какую работу страничный кеш делает за приложение.

152. Практика 11: простой журнал только с дописью в конец

```
допись записи в журнал
↓
write()
↓
синхронизация по необходимости
↓
подтверждение
```

Что делаем

Сделать минимальный журнал с дописью в конец.

Сравнить подтверждение сразу после write() и после Sync().

Описать гарантии каждого режима.

В учебной практике не нужно физически отключать питание накопителя.

153. Практика 12: PostgreSQL WAL

```
транзакция
↓
WAL
↓
сброс с гарантией сохранения
↓
задержка COMMIT
```

Что делаем

В лабораторной PostgreSQL провести много коротких транзакций.

Измерить задержку COMMIT, активность WAL и I/O через iostat.

Затем добавить пакетирование.

Групповой COMMIT снижает стоимость гарантированного сохранения на транзакцию.

154. Практика 13: ограничение ввода-вывода контейнера

```
io.max
```

Что делаем

В отдельном контейнере ограничить I/O через io.max.

Запустить ту же запись, измерить задержку и Dirty.

Проверить, как искусственный лимит превращается в очередь.

Лимит ввода-вывода cgroup может стать узким местом раньше SSD.

155. Практика 14: overlayfs

запись в изменяемый слой образа

Что делаем

В контейнере сравнить запись в изменяемый слой образа и отдельный том.

Измерить задержку, I/O и возможный copy-up.

Топология файловой системы контейнера влияет на путь записи.

156. Практика 15: медленный `fsync()` без высокой пропускной способности

```
мелкая допись
↓
fsync()
```

Что делаем

Сделать много небольших записей с fsync().

Объем данных в MB/s небольшой, но COMMIT может быть медленным.

Сравнить с пакетной синхронизацией.

Задержка надежного сброса важнее общей пропускной способности для маленьких транзакций.

157. Плохое упрощение: `write()` вернулся — SSD записал

```
буферизованная запись
↓
страничный кеш
↓
измененная страница
↓
возврат
```

Где ломается модель

После обычного write() данные могут находиться лишь в измененных страницах RAM.

Для сохранности нужен явно определенный протокол синхронизации.

Успешный write() - не подтверждение записи на энергонезависимый носитель.

158. Плохое упрощение: `fsync()` — просто более медленный `write()`

```
копирование данных
в память ядра
```

Где ломается модель

write() может закончиться после изменения кеша ядра.

fsync() требует завершить обязательства по сохранению данных и метаданных.

write() и fsync() подтверждают разные этапы записи.

159. Плохое упрощение: быстрый SSD делает `fsync()` бесплатным

Плохая модель

Даже быстрый NVMe имеет:

очереди;

протокол;

контроллер;

Более точная модель

FTL;

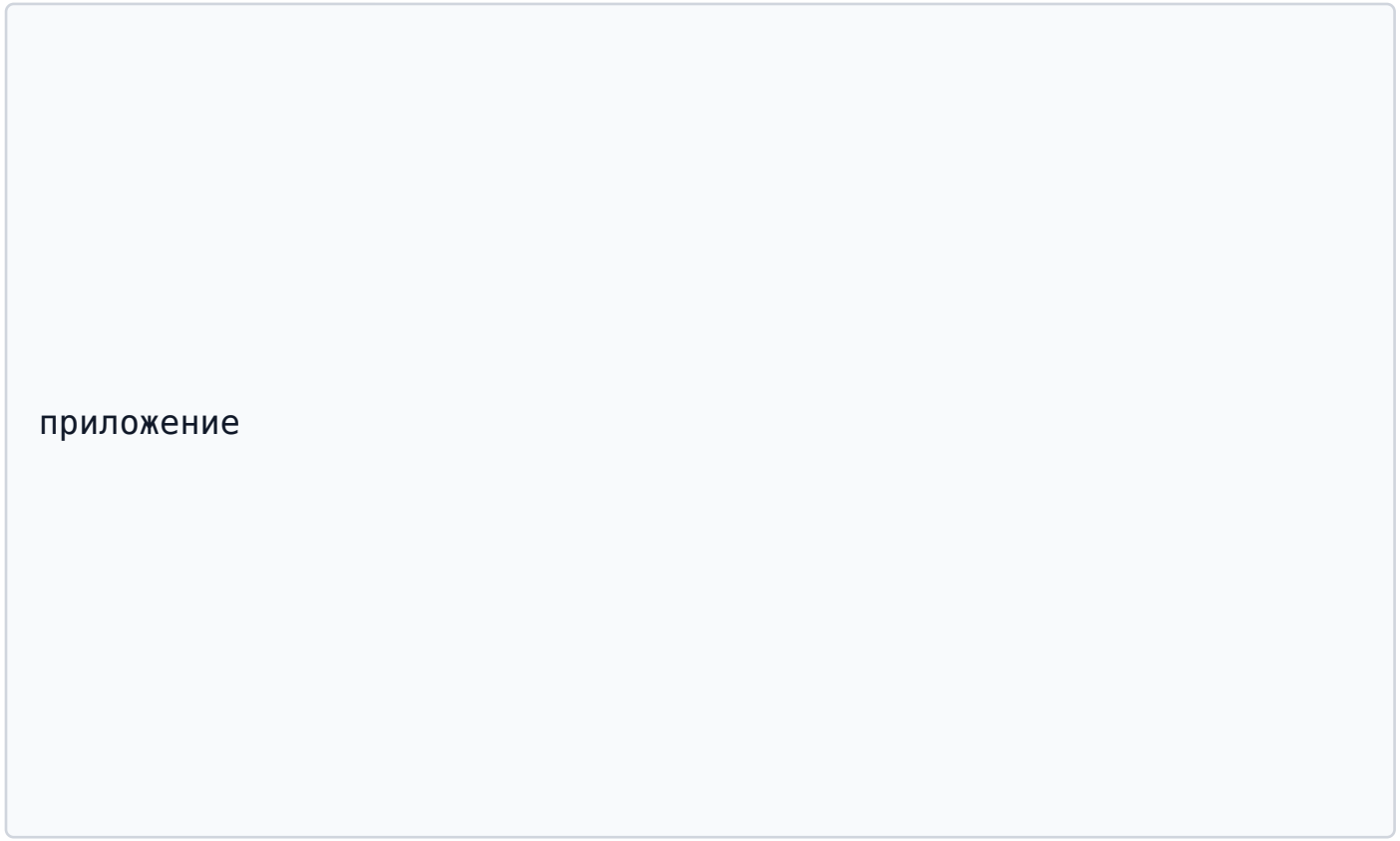
внутренняя сборка мусора;

сброс и порядок записи.

fsync() ждет выполнения гарантий сохранения.

Микросекунды все равно существуют.

160. Ошибка: прямой ввод-вывод всегда быстрее



Где ломается модель

Прямой ввод-вывод исключает обычный страничный кеш.

Но приложение само отвечает за буферизацию, пакетирование и выравнивание.

На мелкой нерегулярной нагрузке буферизованный ввод-вывод может быть быстрее.

161. Плохое упрощение: NVMe = очередей нет

много очередей отправки
много очередей завершений
большая глубина очереди
параллельные команды

Где ломается модель

Преимущество NVMe - не отсутствие очередей, а масштабируемая многоочередная архитектура.

NVMe ускоряется благодаря параллелизму, а не устранению очередей.

162. Плохое упрощение: `%util = 100` значит NVMe полностью исчерпан

```
await
глубина очереди
IOPS
пропускная способность
p99
```

Где ломается модель

На многоочередном NVMe показатель %util требует осторожной интерпретации.

Дополнительно смотрим await, глубину очереди, IOPS, поток данных и p99.

Одной метрики %util недостаточно для диагноза насыщения NVMe.

163. Плохое упрощение: SLC-кеш = реальная скорость NAND

```
прямая запись в TLC/QLC
+
перенос данных из SLC-кеша
+
сборка мусора SSD
```

Где ломается модель

SLC-кеш ускоряет кратковременные всплески записи.

Когда он заполнен:

скорость может радикально измениться.

164. Плохое упрощение: прошивка SSD — невидимая деталь

корректность прошивки

Где ломается модель

Прошивка управляет размещением данных и гарантиями накопителя.

Если она нарушает контракт сохранения, БД тоже может потерять корректность.

Корректность прошивки SSD - часть надежности системы хранения.

165. Плохое упрощение: WAL — просто лог для аудита

```
последовательный допись в конец + сброс
```

Где ломается модель

WAL обеспечивает сохранность и восстановление транзакций.

Допись и групповой сброс часто дешевле синхронной записи множества случайных страниц.

WAL - механизм надежного хранения, а не просто журнал для аудита.

166. Плохое упрощение: высокий страничный кеш — плохо

неконтролируемый рост
измененных страниц

Где ломается модель

Страничный кеш - нормальное использование RAM.

Проблема возникает при постоянном росте Dirty, давлении на память и замедлении записи.

Большой файловый кеш сам по себе не является неисправностью.

167. Плохое упрощение: `Dirty` всегда должен быть около нуля

```
пакетирование
+
объединение записей
+
асинхронная фоновая запись
```

Где ломается модель

Нет.

Небольшой очередь измененных страниц нормален.

Он позволяет:

Проблема — устойчивый рост и throttling.

168. Ошибка: fsync() после каждой записи бесплатен

```
один fsync()
на каждую запись
↓
малая пропускная способность
```

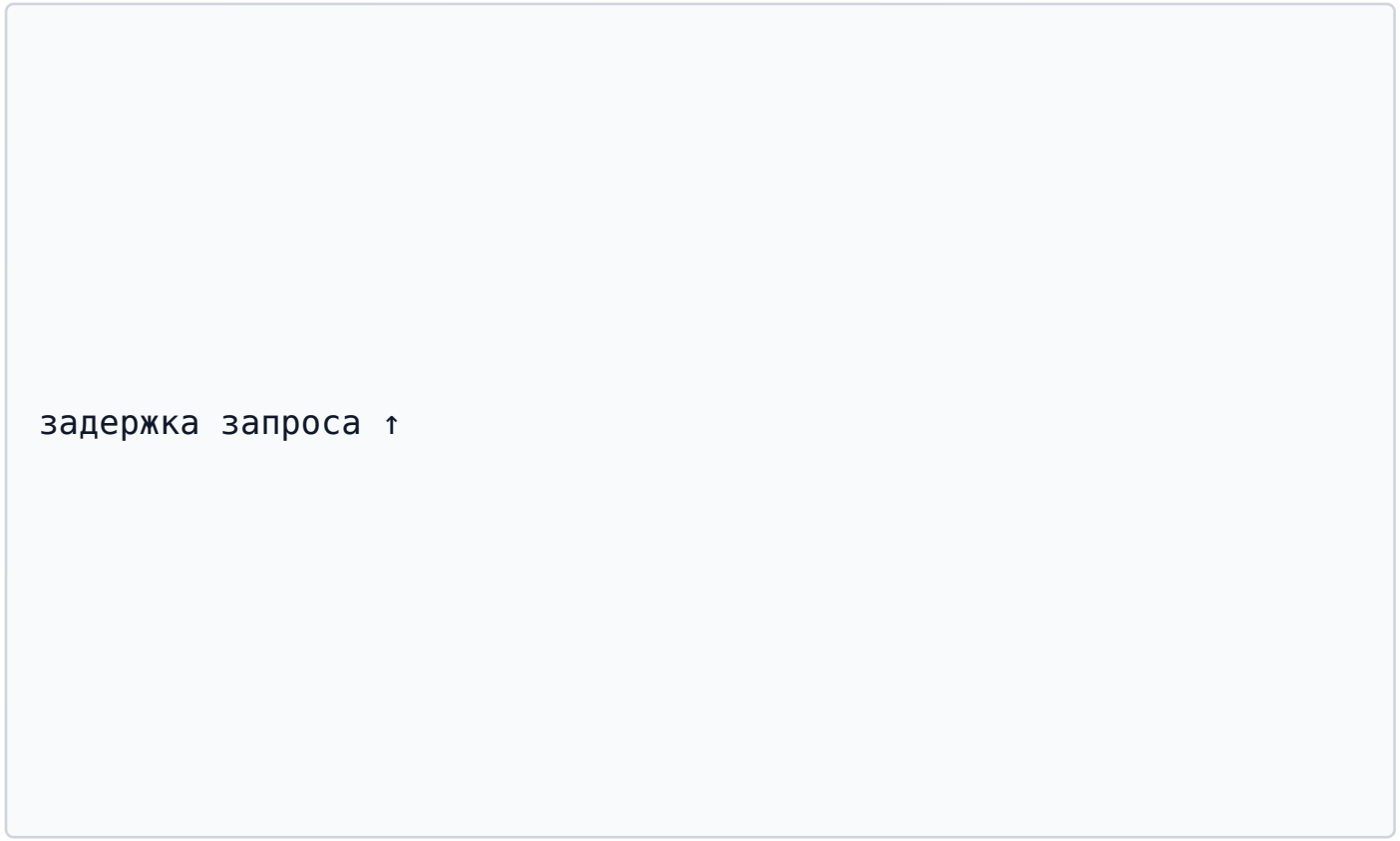
Где ломается модель

Гарантии сохранения становятся проще, но цена каждой операции остается.

При высокой частоте записей полезны пакетирование и групповой COMMIT.

Надежность требует протокола, а пропускная способность - разумного объединения сбросов.

169. Как диагностировать инцидент с накопителем сверху вниз



Механизм

Начните с измерения задержки приложения.

Проверьте системные вызовы, страничный кеш, ФС, блочный слой и SSD.

Не стоит начинать сразу с NAND.

Диагностика накопителя идет от наблюдаемого симптома вниз по слоям.

170. Уровень 1: системный вызов

```
strace
трассировка/eBPF
приложение замеры времени
```

Что измеряем

Первый вопрос: где именно ждет обработчик?

Измеряем время write()/fsync() через приложение, strace и eBPF.

Если ожидание совпадает с системным вызовом, связь уже найдена.

Где обработчик реально ждет?

171. Уровень 2: страничный кеш и фоновая запись

```
Dirty
Writeback
задержка записи
счетчики writeback
```

Что измеряем

Вопрос:

Смотрим:

Storage отстает от изменение страниц?

172. Уровень 3: файловая система

- Вопросы:
- метаданные давление?
- COMMIT журнала?
- ENOSPC?
- фрагментация?
- overlay слой?
- Здесь уже нужны специфичные для файловой системы инструменты и точки трассировки.

отложенное выделение блоков?

173. Уровень 4: блочный слой

```
iostat
точки трассировки блочного слоя
трассировка блочного слоя/eBPF
```

Что измеряем

На уровне блочного слоя ищем очередь перед устройством.

Смотрим iostat и точки трассировки запросов/завершений.

Хостовые очереди и время устройства нужно отличать от ожидания в приложении.

174. Уровень 5: устройство

```
внутренняя сборка мусора SSD?
SLC-кеш исчерпан?
температурное ограничение устройства?
прошивка?
износ?
```

Что измеряем

Если со стороны Linux путь выглядит нормально, а завершения нестабильны, проверяем SSD:

сборку мусора, SLC-кеш, температуру, прошивку и износ.

Нестабильная задержка может возникать внутри самого накопителя.

175. Температурное ограничение производительности SSD

```
температура ↑
↓
контроллер принудительно замедляет
↓
пропускная способность ↓
↓
задержка ↑
```

Механизм

При длительной записи NVMe нагревается.

Контроллер может снижать скорость для защиты от перегрева.

Поэтому короткий и длительный тесты отличаются.

Терморегулирование SSD может ухудшать длительную производительность.

176. SMART и состояние NVMe

- Полезно смотреть:
- температура;
- процент износа;
- ошибки носителя;
- доступный резерв;
- записанные блоки данных;
- аварийные выключения;
- журнал ошибок.

Это уже устройство, а не Linux страничный кеш.

177. Почему аварийные выключения важны

```
состояние контроллера
метаданные FTL
энергозависимый кеш
```

Механизм

После внезапной потери питания могут пострадать данные в кеше, метаданные FTL и состояние контроллера.

PLP и корректная прошивка уменьшают риск.

Данные накопителя нужно защищать от внезапного отключения питания.

178. Как файловая система и SSD взаимодействуют через discard

```
LBA все еще содержит полезные данные
```

Механизм

Файловая система освобождает логические блоки.

TRIM/discard сообщает накопителю, что их данные больше не нужны.

Контроллер может избежать лишнего копирования.

TRIM помогает SSD сокращать внутреннюю работу по переносу ненужных данных.

179. Почему discard не означает мгновенное стирание NAND

когда реально использовать эту информацию

Механизм

TRIM сообщает, что логические блоки больше не нужны.

Контроллер сам решает, когда физически стереть NAND.

Освобождение LBA и физическое стирание ячеек происходят в разное время.

180. Что происходит при всплеске нагрузки

```
всплеск запросов
↓
всплеск записей
↓
страничный кеш поглощает поток
↓
Dirty растет
↓
приложение пока работает быстро
```

Механизм

Если всплеск закончился:

Страничный кеш сработал как буфер.

181. Что происходит под длительной перегрузке

```
скорость записи > скорость SSD
↓
Dirty продолжает расти
↓
достижение порогов
↓
замедление писателей
↓
увеличение очереди
↓
задержка продолжает расти
```

Механизм

Он только отсрочил проявление узкое место.

Буфер не увеличил запаса производительности.

182. Почему это важный бэкенд-паттерн

```
поглощает всплеск
```

Механизм

Буфером может быть страничный кеш, Kafka, RabbitMQ или очередь пула.

Краткий всплеск они поглощают; длительная перегрузка переполняет очередь.

Любой буфер помогает при всплесках, но не устраняет устойчивое узкое место.

183. Что происходит при синхронной нагрузке

```
запись
↓
сброс
↓
ожидание
↓
следующая запись
```

Механизм

При последовательных синхронных записях пропускная способность ограничена задержкой одной операции.

Помогают пакетирование или параллелизм, но меняют режим ожидания.

Малая глубина очереди делает задержку fsync() ключевым ограничителем.

184. Почему слишком большая параллелизм ввода-вывода вреден

```
1 → 64 одновременных писателя
```

Механизм

Больший параллелизм может поднять пропускную способность SSD.

Но очереди и хвост задержки тоже могут вырасти, ухудшив p99 бэкенда.

Максимум пропускной способности не равен минимуму p99.

185. Хвост задержки и справедливое распределение

```
встает за массового ввода-вывода
```

Механизм

Приоритетный fsync() журнала может оказаться за массивной фоновой записью.

Планировщик ввода-вывода и политика ресурсов нужны для справедливости.

Фоновая запись может задерживать критический fsync() в общей очереди.

186. `ionice` и политика ввода-вывода

```
фоновый массовый ввод-вывод
```

Механизм

ionice задает приоритет операций ввода-вывода, но реальный эффект зависит от планировщика и cgroup.

Фоновая загрузка не должна безусловно иметь равный приоритет с критическим WAL.

Эффект приоритетов нужно проверять для конкретного ядра и планировщика.

187. Изоляция ввода-вывода через cgroup

```
io.weight
io.max
```

Механизм

io.weight задает относительный вес, а io.max - жесткие ограничения скорости/IOPS.

Они управляют конкуренцией за накопитель, но сами создают ожидание.

Контейнерные лимиты меняют очереди и задержку еще до физического SSD.

188. Что будет при слишком жестком `io.max`

```
лимит ввода-вывода замедляет записи
↓
фоновый сброс отстает
↓
Dirty растет
↓
пишущие задачи ждут
↓
p99 растет
```

Механизм

Ограничили накопитель — получили память/writeback эффект выше по стек.

Приложение способно изменять страницы быстрее разрешенного I/O.

189. Память и накопитель снова замкнулись

```
скорость ввода-вывода ограничена
↓
фоновая запись отстает
↓
Dirty занимает RAM
↓
меньше страниц можно освободить
↓
давление на память растет
↓
задержка приложения растет
```

Механизм

Цепочка:

Storage узкое место может стать память узкое место.

190. А давление на память может стать узкое место накопителя

```
RAM давление
↓
измененные страницы нужно записать до их освобождения
↓
more ввод-вывод накопителя
↓
устройство давление
↓
задержка ↑
```

Механизм

Обратная связь: нехватка RAM заставляет ядро записывать измененные страницы ради освобождения памяти.

Это добавляет нагрузку на SSD.

Давление на память способно усилить перегрузку накопителя.

191. Почему продовый инцидент редко живет в одном слое

```
лимит памяти контейнера уменьшили
↓
для страничного кеша осталось меньше места
↓
больше чтений промахиваются мимо кеша
↓
накопитель чтения ↑
↓
DB задержка ↑
↓
число одновременных запросов ↑
↓
давление на память ↑
```

Механизм

Например, уменьшили лимит памяти контейнера.

Страничного кеша стало меньше, чтения чаще идут на SSD, БД отвечает медленнее, запросов в работе больше, давление на память растет.

Один инфраструктурный лимит может запустить цепочку с памятью, диском и очередями бэкенда.

192. Гарантии спецификации, API и реализации

```
запись
fsync
fdatasync
O_SYNC
```

Механизм

Системные вызовы write(), fsync(), fdatasync() дают документированный API.

Файловая система, блочный слой и контроллер реализуют гарантии на своих уровнях.

Разделяйте контракт системного вызова и детали текущей реализации хранения.

193. Нельзя строить корректность на предположении «SSD обычно пишет быстро»

контракт `fsync()`

Механизм

Производительность SSD и корректность контракта fsync() - разные свойства.

Нельзя делать вывод о сохранности данных из того, что накопитель обычно быстр.

Корректность опирается на гарантии протокола, а не на среднюю скорость устройства.

194. Нельзя путать согласованность после сбоя и гарантированное сохранение

01

Долговечность:

Долговечность означает переживание сбоя подтвержденными данными.
Согласованность после сбоя означает допустимое состояние структур хранения.
Обе задачи нужны, но решаются разными механизмами.

Согласованность после сбоя (crash согласованность):

03

После сбой структура данных остается в допустимом состоянии.

04

Журнал файловой системы и WAL решают похожие, но не идентичные задачи на разных уровнях.

Подтвержденные данные переживают сбой согласно contract.

195. WAL не заменяет файловую систему

- Файловая система обеспечивает:
- пространство имен файлов;
- отображение блоков;
- согласованность метаданных;
- доступ к накопителю.

- WAL базы данных опирается на гарантии файловой системы и устройства.

WAL обеспечивает протокол восстановления базы.

196. Журнал файловой системы не заменяет WAL базы данных

метаданным файловой системы

Механизм

Журнал ФС знает свои блоки и метаданные, но не бизнес-транзакции PostgreSQL.

Он не заменяет WAL СУБД.

Журнал ФС и WAL СУБД обеспечивают согласованность на разных уровнях.

197. Финальная причинная цепочка `write()`

```
приложение
↓
write()
↓
копирование в страничный кеш
↓
учет измененных страниц
↓
возврат из write()
↓
фоновая запись
↓
файловая система
↓
блочные запросы
↓
NVMe
↓
контроллер SSD и FTL
↓
NAND
```

Смотрим на механизм, цену и симптом под нагрузкой.

198. Финальная причинная цепочка `fsync()`

```
приложение
↓
fsync()
↓
сброс измененного состояния файла
↓
файловая система завершает запись журнала и нужных метаданных
↓
блочный ввод-вывод
↓
дождаться завершений
↓
Flush/FUA при необходимости
↓
семантика гарантированного сохранения устройства
↓
возврат
```

Смотрим на механизм, цену и симптом под нагрузкой.

199. Финальная причинная цепочка WAL

```
транзакция
↓
логические изменения
↓
запись WAL
↓
допись в конец
↓
групповой COMMIT (`group commit`)
↓
fsync
↓
гарантированно сохраненный WAL
↓
подтверждение COMMIT
↓
страницы данных позже
```

Смотрим на механизм, цену и симптом под нагрузкой.

Что важно запомнить · 1

- Успешный write() обычно означает, что ядро приняло данные, но еще не гарантирует изменение NAND.
- Страничный кеш скрывает задержку накопителя и объединяет операции ввода-вывода, но создает разрыв между записью и гарантированным сохранением.
- Измененные страницы — это накопленная очередь работы для подсистемы хранения.
- Если приложение изменяет страницы быстрее, чем накопитель выполняет устойчивую фоновую запись, очередь растет и Linux начинает замедлять пишущие процессы.
- Давление на память и writeback связаны: измененные страницы нельзя просто освободить без сохранения их содержимого.
- fsync() задает границу гарантированного сохранения, а не является просто «еще одной записью».
- fdatasync() имеет более узкий контракт, ориентированный на данные, но выигрыш зависит от файловой системы.
- WAL превращает множество случайных гарантированно сохраняемых обновлений в компактный последовательный поток записи.
- Групповой COMMIT (групповой COMMIT) распределяет стоимость одного сброса с гарантией сохранения между несколькими транзакциями.
- Журнал файловой системы и WAL базы данных решают разные задачи согласованности и восстановления.

Сводим путь записи в причинные связи, а не набор терминов.

Что важно запомнить · 2

- Отложенное выделение блоков улучшает размещение, но часть ошибок может проявиться позднее — во время writeback или fsync().
- close() не является универсальной заменой fsync().
- прямой ввод-вывод обходит обычный страничный кеш данных, но не убирает все кеши из пути до накопителя.
- blk-mq масштабирует блочный ввод-вывод через многоочередную архитектуру.
- Глубина очереди — ограниченный ресурс: она может увеличить пропускную способность, но одновременно увеличивает задержку ожидания.
- NVMe использует очереди отправки и завершения поверх PCIe; очереди и накладные расходы протокола никуда не исчезают.
- Контроллер SSD скрывает FTL, внутреннюю сборку мусора, выравнивание износа и собственное планирование.
- NAND требует записи в новое физическое место и стирания более крупными блоками, поэтому SSD нельзя представлять как «RAM, которая не забывает».
- Внутренняя сборка мусора SSD создает усиление записи и всплески хвостовой задержки.
- SLC-кеш способен сильно завысить результаты короткого теста потребительского SSD по сравнению с длительной нагрузкой.

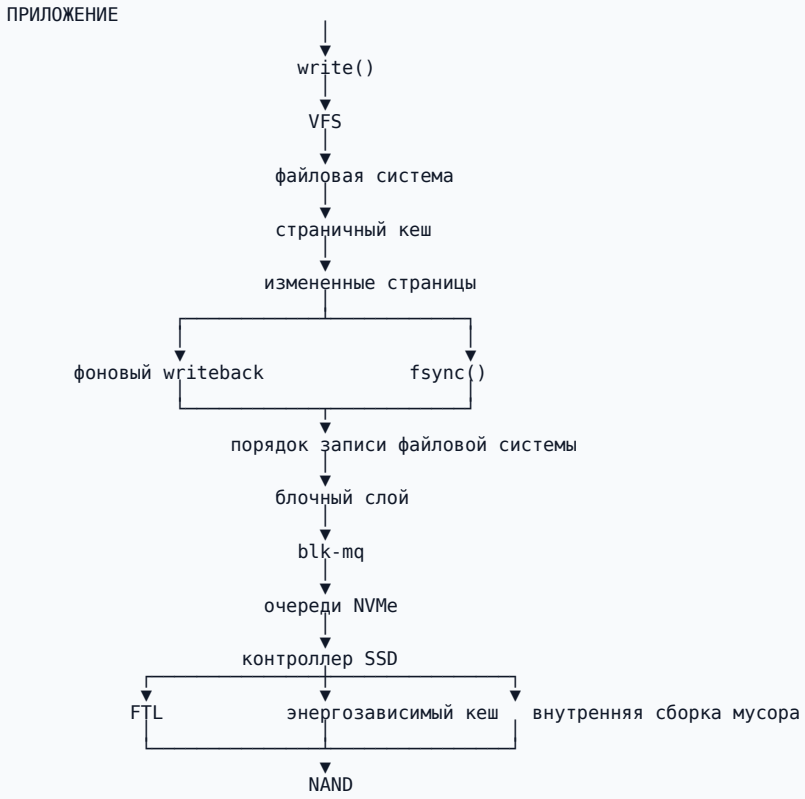
Сводим путь записи в причинные связи, а не набор терминов.

Что важно запомнить · 3

- Flush/FUA нужны потому, что устройство может иметь энергозависимый кеш записи.
- PLP — часть инженерии гарантированного сохранения данных на накопителе.
- Последовательная пропускная способность не описывает задержку fsync() при QD1.
- Высокий IOPS из спецификации устройства не переносится автоматически на нагрузку WAL/OLTP.
- Диагностику нужно вести сверху вниз: системный вызов → страничный кеш/writeback → файловая система → блочный слой → устройство.
- strace, iostat, /proc/meminfo, pidstat, fio, eBPF и телеметрия устройства отвечают на разные вопросы.
- Overlayfs контейнера и лимиты ввода-вывода cgroup могут добавить задержку еще до физического устройства.
- Перегрузка накопителя может перерасти в давление на память через накопление измененных страниц.
- Давление на память может перерасти в нагрузку на накопитель через освобождение страниц и writeback.
- Реальная стоимость гарантированной записи определяется всей цепочкой от пользовательского пространства до энергонезависимого носителя.

Сводим путь записи в причинные связи, а не набор терминов.

Финальная модель · 1



От write() до NAND - цепочка кешей, очередей и протоколов.

Финальная модель · 2

```
write() вернулся
↓
ядро приняло данные

фоновая запись завершена
↓
операция I/O завершена

fsync() вернулся
↓
контракт сохранности ФС и накопителя выполнен
```

Фоновая запись и fsync() подтверждают разные этапы.

Финальная модель · 3

```
скорость записи растет
↓
очередь измененных страниц растет
↓
фоновая запись перегружена
↓
ядро замедляет писателей
↓
очереди блочного слоя растут
↓
задержка растет
↓
p99 увеличивается
```

Под перегрузкой буфер превращается в очередь, а очередь - в задержку.

Финальная модель · 4

```
транзакция
↓
WAL
↓
пакет
↓
fsync
↓
гарантированное сохранение NVMe/SSD
↓
ACK
```

WAL переносит гарантированное сохранение транзакции на последовательный журнал.

Главный инженерный вывод

Путь записи

Под нагрузкой:

Страничный кеш сглаживает записи, пока SSD успевает. Если поток превышает скорость накопителя, Dirty и задержка растут.

Для транзакционной системы:

WAL и групповой COMMIT распределяют цену fsync() между транзакциями.

Инженерный вывод: write(), фоновая запись и fsync() подтверждают разные этапы работы с данными.

Ядро приняло данные → I/O завершилось → данные переживут сбой.

Основные источники

- Linux man-pages: `write(2)`, `fsync(2)`, `fdatasync(2)`, `open(2)`, `mmap(2)`, `msync(2)`
- Linux Kernel Documentation — VM writeback settings: https://docs.kernel.org/admin-guide/sysctl/vm.html
- Linux Kernel Documentation — blk-mq: https://docs.kernel.org/block/blk-mq.html
- Linux Kernel Documentation — block layer: https://docs.kernel.org/block/
- Linux Kernel Documentation — filesystems: https://docs.kernel.org/filesystems/
- Linux Kernel Documentation — ext4: https://docs.kernel.org/filesystems/ext4/
- Linux Kernel Documentation — XFS: https://docs.kernel.org/filesystems/xfs/
- NVM Express Specifications: https://nvmexpress.org/specifications/
- fio documentation: https://fio.readthedocs.io/
- PostgreSQL Documentation — WAL: https://www.postgresql.org/docs/current/wal.html